KarigarUstaad

Final Year Project

Session 2022-2026

A project submitted in partial fulfillment of the degree of

BS in Software Engineering

Department of Software Engineering

Faculty of Computer Science & Information Technology

The Superior University, Lahore

Fall 2025

Type	[✓] **D**evelopment [ ] **R**esearch [ ] **R&D**
Area of specialization	App Development
FYP ID	BSSE-FYP-F25-022

Project Group Members

Sr.#	Reg. #	Student Name	Email ID	*Signature
(i)	F22-067	Abdul Wahab Virk	su92-bssem-f22-067@superior.edu.pk	
(ii)	F22-069	Muhammad Saad	su92-bssem-f22-069@superior.edu.pk	
(iii)	F22-246	Ayyan Shameen	su92-bssem-f22-246@superior.edu.pk	

*The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to work of others

Plagiarism Free Certificate

This is to certify that, I Abdul Wahab Virk Son of Saif Ullah Virk, group leader of FYP under registration no BSSE-FYP-F25-022 at Software Engineering Department, The Superior University, Lahore. I declare that my FYP report is checked by my supervisor.

Date: _____ Name of Group Leader: Abdul Wahab Virk Signature: _____

Name of Supervisor: _____ Co-Supervisor: _____

Designation: _____ Designation: _____

Signature: _____ Signature: _____

HoD: Dr. Tehreem Masood

Signature: _____

KarigarUstaad

Change Record

Author(s)	Version	Date	Notes	Supervisor's Signature
Abdul Wahab Virk Muhammad Saad Ayyan Shameen	1.0	10-10-2025	Initial draft prepared; Chapters 1 and 2 completed with problem definition, objectives, and scope refinement	
Abdul Wahab Virk Muhammad Saad Ayyan Shameen	1.1	17-10-2025	UI/UX design finalized; low-fidelity and high-fidelity screens improved based on usability considerations	
Abdul Wahab Virk Muhammad Saad Ayyan Shameen	1.2	24-10-2025	System diagrams completed and refined; use case, class, ERD, and sequence diagrams improved for clarity and consistency	
Abdul Wahab Virk Muhammad Saad Ayyan Shameen	1.3	02-11-2025	Chapters 3 and 4 enhanced; system design, architecture, and requirements reviewed and updated	
Abdul Wahab Virk Muhammad Saad Ayyan Shameen	1.4	10-11-2025	Promotional materials designed and improved, including standee, flyer, brochure, and banner for project presentation	
Abdul Wahab Virk Muhammad Saad Ayyan Shameen	1.5	17-11-2025	Chapter 5 completed; implementation details refined and aligned with MERN stack architecture	
Abdul Wahab Virk Muhammad Saad Ayyan Shameen	1.6	27-11-2025	Final review performed; formatting, content consistency, diagrams, and overall documentation improved	

APPROVAL

PROJECT SUPERVISOR

Comments: _____

_____

Name: Syed Zeeshan Hussain

Date:_____ Signature:_____

PROJECT MANAGER

Comments: _____

_____

Date:_____ Signature:_____

HEAD OF THE DEPARTMENT

Comments: _____

_____

Date:_____ Signature:_____

Dedication

This project is dedicated to the individuals and communities who face the challenges that inspired the development of this system. Their needs, experiences, and daily struggles motivated us to design a solution that contributes meaningfully to addressing their problems. We hope that our efforts in this Final Year Project not only acknowledge their realities but also serve as a step toward improving their quality of life.

Acknowledgements

We would like to express our sincere gratitude to our supervisor for their continuous guidance, valuable insights, and encouragement throughout the duration of this Final Year Project. Their support played a crucial role in shaping our understanding and strengthening the quality of our work.

We are also thankful to our department, faculty members, and peers for providing an environment that fostered learning, collaboration, and growth. Finally, we extend our appreciation to all individuals users, stakeholders, and domain experts who contributed their time, feedback, and perspectives, enabling us to develop a solution that is both practical and impactful.

Executive Summary

KarigarUstaad is a comprehensive digital service marketplace designed to address the growing need for reliable, verified, and timely home and business maintenance services in Pakistan. As urbanization accelerates and new housing societies continue to expand, consumers and businesses face significant challenges in accessing trustworthy electricians, plumbers, technicians, painters, and other skilled professionals. The market currently suffers from fragmented service availability, lack of transparent pricing, inconsistent service quality, and minimal worker verification, leaving users uncertain and underserved. KarigarUstaad fills this gap by offering an all-in-one platform that provides convenience, credibility, and control.

The platform operates on a dual-service model. Users can choose between third-party freelancers and the company's own trained, in-house workforce. This approach caters to both price-sensitive customers seeking cost-effective options and quality-focused users who prefer standardized and warrantied services. By incorporating real-time tracking, secure payment gateways, transparent pricing, and detailed rating systems, the platform ensures a seamless user experience while promoting accountability and trust. KarigarUstaad's B2B functionality further extends its value by empowering offices, small enterprises, and property managers with dependable, on-demand maintenance and repair solutions.

KarigarUstaad not only addresses local needs but also has the capacity to expand into other developing countries facing similar challenges. Its revenue model which includes service commissions, in-house workforce earnings, premium bookings, featured listings, and subscription options offers a sustainable path to long-term growth and profitability.

KarigarUstaad aims to redefine service accessibility by combining technology, transparency, and professional reliability. By bridging the gap between consumers and trusted service providers, it offers a dependable, modern, and scalable solution poised to transform the skilled-services marketplace.

Table of Contents

Chapter 1 1
Introduction 1
1.1. Background 2
1.2. Motivations and Challenges 2
1.3. Goals and Objectives 2
1.4. Literature Review/Existing Solutions 3
1.5. Gap Analysis 3
1.6. Proposed Solution 3
1.7. Project Plan 4
1.7.1. Work Breakdown Structure 4
1.7.2. Roles & Responsibility Matrix 5
1.7.3. Gantt Chart 6
1.8. Report Outline 7
1.9. Empathy Map 7
Chapter 2 9
Software Requirement Specifications 9
Chapter 2: Software Requirement Specifications 10
2.1. Introduction 10
2.1.1. Purpose 10
2.1.2. Document Conventions 10
2.1.3. Intended Audience and Reading Suggestions 10
2.1.4. Product Scope 11
2.1.5. References 11
2.2. Overall Description 11
2.2.1. Product Perspective 11
2.2.2. User Classes and Characteristics 12
2.2.3. Operating Environment 12
2.2.4. Design and Implementation Constraints 12
2.2.5. Assumptions and Dependencies 13
2.3. External Interface Requirements 13
2.3.1. User Interfaces 13
2.3.2. Hardware Interfaces 19
2.3.3. Software Interfaces 19
2.3.4. Communications Interfaces 19
2.4. System Features 19
2.4.1. User Registration & Authentication 19
2.4.1.1. Description and Priority 19
2.4.1.2. Stimulus/Response Sequences 19
2.4.1.3. Functional Requirements 20
2.4.2. Service Booking & Scheduling 20
2.4.2.1. Description and Priority 20
2.4.2.2. Stimulus/Response Sequences 20
2.4.2.3. Functional Requirements 20
2.4.3. AI-Powered Matching & Dynamic Pricing 20
2.4.3.1. Description and Priority 20

2.4.3.2. Functional Requirements 20
2.5. Nonfunctional Requirements 21
2.5.1. Performance Requirements 21
2.5.2. Safety Requirements 21
2.5.3. Security Requirements 21
2.5.4. Usability Requirements 21
2.5.5. Reliability Requirements 21
2.5.6. Maintainability/Supportability Requirements 21
2.5.7. Portability Requirements 21
2.5.8. Efficiency Requirements 21
2.6. Domain Requirements 22
Chapter 3 23
Use Case Analysis 23
3.1. Use Case Model 24
3.2. Use Cases Description 28
Chapter 4 35
System Design 35
4.1. Architecture Diagram 36
4.2. Domain Model 37
4.3. Entity Relationship Diagram with data dictionary 38
4.4. Class Diagram 39
4.5. Sequence / Collaboration Diagram 40
4.6. Operation contracts 40
4.7. Activity Diagram 44
4.8. State Transition Diagram 45
4.9. Component Diagram 46
4.10. Deployment Diagram 47
4.11. Data Flow diagram 47
Chapter 5 53
Implementation 53
5.1. Important Flow Control/Pseudo codes 54
5.2. Components, Libraries, Web Services and stubs 56
5.3. Deployment Environment 57
5.4. Tools and Techniques 57
5.5. Best Practices / Coding Standards 58
5.6. Version Control 58
Chapter 6 60
Testing and Evaluation 60
6.1. Use Case Testing 61
6.2. Equivalence partitioning 61
6.3. Boundary value analysis 61
6.4. Data flow testing 62
6.5. Unit testing 62
6.6. Integration testing 62
6.7. Performance testing 62
6.8. Stress Testing 62

Chapter 7 63
Summary, Conclusion and Future Enhancements 63
7.1. Project Summary 64
7.2. Achievements and Improvements 64
7.3. Critical Review 64
7.4. Lessons Learnt 65
7.5. Future Enhancements/Recommendations 65
Appendices 66
Appendix A: Information / Promotional Material 67
Reference and Bibliography 72
Index 74
[A] Technical Documentation 75
[B] UI/UX & Prototype Screens 75
[C] Promotional & Supporting Material 75

List of Figures

Figure 1 Gantt Chart 7
Figure 2 Empathy Map 8
Figure 3 (ADMIN) Use Case Model 25
Figure 4 (Customer) Use Case Model 26
Figure 5 (worker) Use Case Model 27
Figure 6 (AI-SYSTEM) Use Case Model 28
Figure 7 Architecture Diagram 36
Figure 8 Domain Model 37
Figure 9 Entity Relationship Diagram 38
Figure 10 Class Diagram 39
Figure 11 Sequence Diagram 40
Figure 12 Activity Diagram 44
Figure 13 State Transition Diagram 45
Figure 14 Component Diagram 46
Figure 15 Deployment Diagram 47
Figure 16 Data Flow Diagram Level 0 47
Figure 17 Data Flow Diagram Level 1 48
Figure 18 Data Flow Diagram Level 2 (A) 49
Figure 19 Data Flow Diagram Level 2 (B) 50
Figure 20 Data Flow Diagram Level 2 (C) 51
Figure 21 Broacher Front Page 67
Figure 22 Broacher Back Page 68
Figure 23 Flyer 69
Figure 24 Standee 70
Figure 25 Banner 71

List of Tables

Table 1 Roles and Responsibility Matrix 5
Table 2 User Classes and Characteristics 12
Table 3 Use Case 1: User Registration & Authentication 28
Table 4 Use Case 2: Browse Services & View Providers 29
Table 5 Use Case 3: Book a Service 30
Table 6 Use Case 4: Provider Accepts/Rejects Booking 30
Table 7 Use Case 5: Real-Time Tracking 31
Table 8 Use Case 6: Payment Processing 32
Table 9 Use Case 7: Rating & Review 32
Table 10 Use Case 8: Provider Profile Management 33

Chapter 1

Introduction

Chapter 1: Introduction

This chapter introduces the concept and purpose of the KarigarUstaad platform. It outlines the background of service accessibility issues in Pakistan, discusses the motivations behind developing the system, and highlights the existing challenges in the current market. The chapter also presents the project goals, analyzes gaps in available solutions, and describes the proposed approach. Finally, it includes the project plan, work breakdown structure, team responsibilities, Gantt chart overview, report outline, and an empathy map to understand user needs.

1.1. Background

The increasing urbanization of Pakistan has created a growing need for dependable home and business maintenance services. Many individuals struggle to find reliable electricians, plumbers, repair technicians, and other skilled workers in a timely manner. Despite a rising number of online platforms, issues such as inconsistent service quality, unverified workers, unclear pricing, and lack of accountability persist. As digital adoption grows, there is an opportunity to create a structured, technology-enabled marketplace that provides trusted and transparent services to households and businesses. KarigarUstaad was conceptualized to address these challenges through a dual-service model supported by modern technology.

1.2. Motivations and Challenges

The primary motivation behind KarigarUstaad is to solve the long-standing problem of poor accessibility to skilled and verified service providers. Users frequently encounter unreliable workers, last-minute cancellations, and safety concerns. Service providers also face difficulties finding consistent work due to the absence of an organized platform. Key challenges include managing service quality, ensuring workforce verification, maintaining pricing transparency, and building user trust. Additionally, integrating a hybrid model of freelancers and in-house staff requires efficient coordination, training, and monitoring systems to deliver a standardized service experience.

1.3. Goals and Objectives

The main goal of KarigarUstaad is to build an efficient and trustworthy digital marketplace for home and business services. Its objectives include ensuring reliable service delivery, improving

customer satisfaction, and supporting verified professionals with better work opportunities. The project aims to introduce real-time tracking, rating mechanisms, and secure payment options to enhance transparency. Furthermore, objectives include implementing AI-powered matching, dynamic pricing, and fraud detection to increase accuracy, efficiency, and system reliability. The overall objective is to create a scalable platform that can expand beyond local markets.

1.4. Literature Review/Existing Solutions

Existing platforms in Pakistan, such as **Mahir Company** and a few emerging service apps, offer limited-service categories and lack comprehensive quality control. Many rely solely on freelancers without verification, resulting in unpredictable experiences for customers. International models like **TaskRabbit** and **UrbanClap** demonstrate the potential success of structured service marketplaces but are not directly tailored to Pakistan's local context, pricing norms, or safety concerns. A review of market trends shows a rising demand for digital service platforms, yet no solution fully integrates AI features, dual workforce options, and a B2B model in the Pakistani market.

1.5. Gap Analysis

Current platforms fall short in fundamental areas such as workforce verification, real-time tracking, transparent pricing, and service consistency. Most do not offer both freelancer-based and in-house workforce models, restricting customer choice and quality control. Businesses seeking regular maintenance services also lack a dedicated B2B solution with reliable scheduling and accountability. Additionally, there is an absence of smart systems that leverage AI for matching, fraud detection, or dynamic pricing. These gaps highlight the need for a robust, well-regulated platform like KarigarUstaad.

1.6. Proposed Solution

KarigarUstaad proposes a dual-model service platform that connects users with both freelance professionals and trained in-house staff. The platform integrates real-time tracking, verified worker profiles, transparent pricing, and secure payments to enhance trust and convenience. AI-powered features such as intelligent provider matching, automated cost estimation, quality control algorithms, and chatbot-based user support further improve usability. The solution aims

to create a reliable and scalable ecosystem serving home users, business clients, and service providers.

1.7. Project Plan

The project plan outlines the development phases, resource allocation, and timelines required to build KarigarUstaad. It includes system analysis, UI/UX design, backend development, mobile app development, AI integration, testing, deployment, and marketing. The plan emphasizes iterative development using agile methodologies to ensure flexibility and continuous improvement. Team roles are assigned to ensure effective coordination, while scheduling tools such as Gantt charts help monitor progress throughout the project lifecycle.

1.7.1. Work Breakdown Structure

The work breakdown structure (WBS) divides the project into hierarchical tasks:

1. **Requirements Analysis**
    - User research
    - Market study
    - Feature specification
2. **System Design**
    - UI/UX design
    - Database schema
    - Architecture planning
3. **Development**
    - Mobile app (Android/iOS)
    - Backend APIs
    - Admin dashboard
    - AI module development
4. **Testing**
    - Functional testing
    - Performance testing
    - Security testing
5. **Deployment**
    - Server setup
    - App store deployment
    - Initial user onboarding
6. **Marketing & Launch**
    - Branding
    - Social media campaigns
    - Business partnerships

1.7.2. Roles & Responsibility Matrix

Table 1 Roles and Responsibility Matrix

Role	Responsibilities
Project Manager (All members)	All members jointly contributed to project planning, requirement gathering, documentation, design, development, and testing. Responsibilities were shared equally throughout the project lifecycle.
UI/UX Designer (Shared role)	The entire team collaboratively designed wireframes, user flows, and app interface screens in Figma. All design decisions were made collectively.
Backend Developer (Shared role)	Backend development tasks including API development, database schemas, and server logic will be handled collectively during the development phase.
Mobile Developer (Shared role)	All team members will participate in building the MERN stack application frontend (React) and connecting it with backend APIs. Component integration and UI functionality development are still in progress
AI/ML Engineer (Shared role)	The team collectively brainstormed the AI recommendation approach. Actual AI implementation (worker matching & scoring model) is planned for later development phases..
QA Tester (Shared role)	Testing, debugging, and performance evaluation will be performed collaboratively as development continues.
Marketing Lead (Muhammad Saad)	Lead role in marketing strategy, running social media ads, audience targeting, outreach, and promotional planning. Oversees digital marketing activities.
Branding & Creative Team – All Members	Branding & Creative Team – All Members

1.7.3. Gantt Chart

Figure 1 Gantt Chart

1.8. Report Outline

This report is structured into chapters covering the project's introduction, market research, system requirements, design methodology, implementation, testing, and conclusion. Each chapter builds upon the previous one to provide a complete understanding of the problem domain, the proposed solution, and the development process. The outline ensures clarity, logical flow, and comprehensive documentation of the KarigarUstaad platform.

1.9. Empathy Map

User Thinks:
They want a safe, trustworthy, and professional service without hassle. They value transparency and reliability.

User Feels:
Concerned about safety, annoyed by previous bad experiences, and relieved when finding a dependable service.

User Says:
"I need someone reliable," "I want clear pricing," "I don't trust random workers," "I need services quickly."

User Does:
Searches online, asks friends for recommendations, compares prices, checks ratings, and prefers verified professionals for home entry.

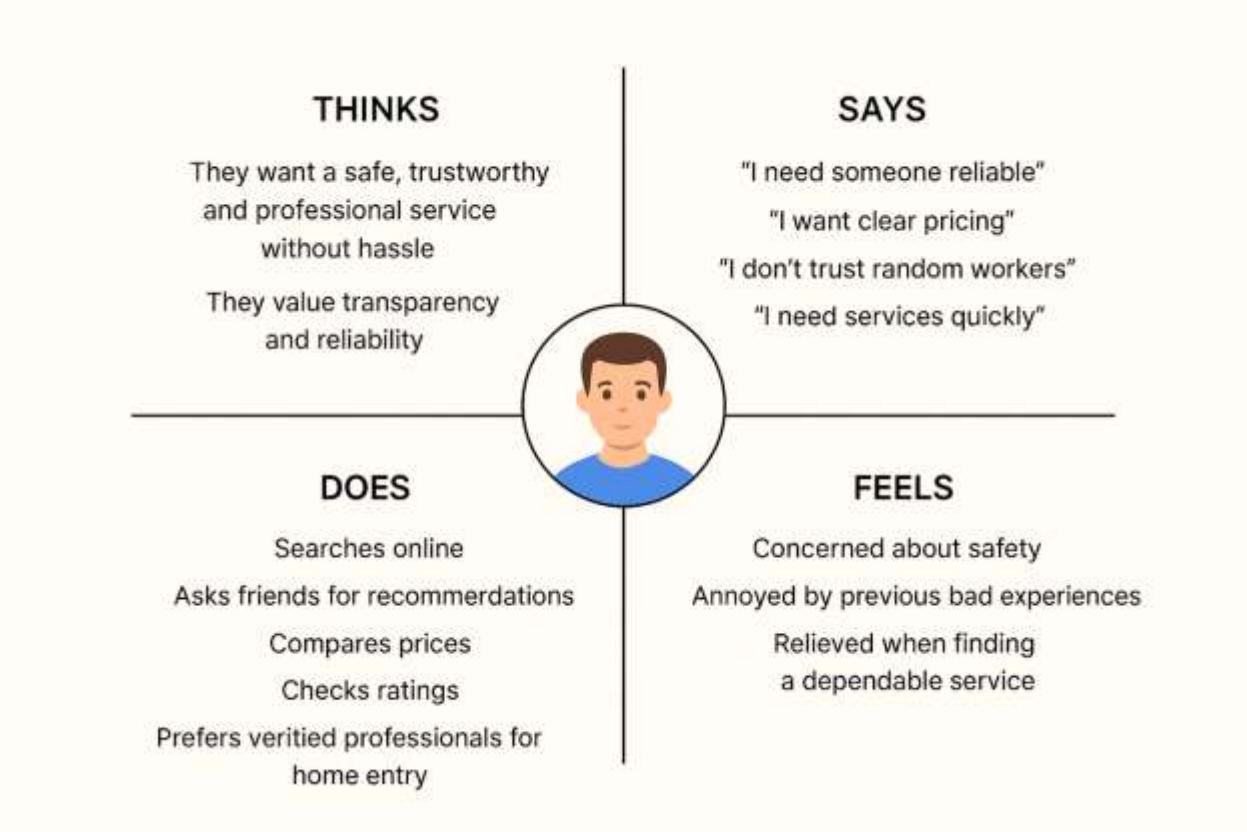


Figure 2 Empathy Map

Chapter 2

Software Requirement Specifications

Chapter 2: Software Requirement Specifications

2.1. Introduction

2.1.1. Purpose

This Software Requirements Specification (SRS) defines the functional and nonfunctional requirements for the KarigarUstaad Service Marketplace Application, Version 1.0. The purpose of this SRS is to outline the complete set of software capabilities needed to develop a dual-model service platform that connects customers with freelance service providers and in-house trained technicians. This document covers the entire system including mobile applications, backend server, admin dashboard, AI-powered features, and integration with third-party services. The SRS does not describe future planned extensions or hardware infrastructure beyond the current scope.

2.1.2. Document Conventions

This SRS follows standard IEEE formatting conventions.

- **Font:** Calibri, 12 pt
- **Headings:** Bold
- **Line and Paragraph Spacing:** 1.5
- **Requirement IDs:** REQ-<Feature><Number> (e.g., REQ-SF1-1)
- **Priorities:** High (H), Medium (M), Low (L)
- **TBD:** “To Be Decided” wherever exact information is pending
- Functional requirements are numbered uniquely and grouped under their respective system features.

2.1.3. Intended Audience and Reading Suggestions

This document is intended for:

- **Developers:** to understand system behavior and interfaces
- **Project Managers:** to plan development timelines and resources
- **Testers:** to design test cases based on functional specifications
- **UI/UX Designers:** to understand interface requirements
- **Stakeholders & Management:** to evaluate if the proposed solution meets business goals

Readers should begin with the introduction, followed by the overall description, then system features, and finally non-functional and domain requirements.

2.1.4. Product Scope

KarigarUstaad is a mobile-based service marketplace that enables users to book electricians, plumbers, technicians, and other skilled workers. It supports two operational models: a B2B freelancer marketplace and a company-owned in-house workforce. The system aims to solve issues related to unreliable workers, inconsistent pricing, and lack of verification. Its purpose aligns with the company's broader strategy of digitizing the skilled-services economy in Pakistan and future regional markets.

2.1.5. References

1. IEEE SRS Template - IEEE Std 830-1998
2. Android Developer Guidelines - Google Inc.
3. Apple Human Interface Guidelines - Apple Inc.
4. RESTful API Design Standards - OpenAPI
5. TaskRabbit and UrbanClap market analysis reports (2023)

2.2. Overall Description

2.2.1. Product Perspective

KarigarUstaad is a standalone product but operates as part of a broader service ecosystem involving customers, service providers, business clients, payment gateways, and an admin management system. It is a new application, not a replacement system. Major components include:

- Customer Mobile App
- Service Provider App
- Admin Dashboard
- Backend System & API
- AI Recommendation Engine
- Payment Integration

2.2.2. User Classes and Characteristics

Table 2 User Classes and Characteristics

User Class	Characteristics	Importance
Customers	Use app occasionally, low technical expertise	High
Freelance Service Providers	Moderate expertise, require job bidding and scheduling	High
In-house Technicians	Trained staff, follow company standards	Medium
Business Clients (B2B	Require recurring service, scheduling, invoicing	High
Administrators	Manage data, users, disputes	High
Marketing Staff	Monitor analytics, promotions	Medium

2.2.3. Operating Environment

Mobile Platforms: Android and iOS

Database: MongoDB

AI Engine: Python-based ML microservices

Network: 4G/5G/Wi-Fi

Compatible Browsers (Admin Panel): Chrome

2.2.4. Design and Implementation Constraints

- Must comply with Pakistan's digital payment regulations
- Must use secure HTTPS for all communication
- Backend must support scalable microservices architecture
- Technology Constraints:
    - Mobile apps: React Native
    - Backend: Node.js
- AI modules must remain lightweight to ensure low latency
- Limited availability of verified service data for training AI

2.2.5. Assumptions and Dependencies

- Users have stable internet access
- Third-party payment gateways (e.g., JazzCash, EasyPaisa) will remain operational
- Google Maps API will be available for location services
- Service providers will comply with verification protocols
- Future system updates depend on company's operational expansion

2.3. External Interface Requirements

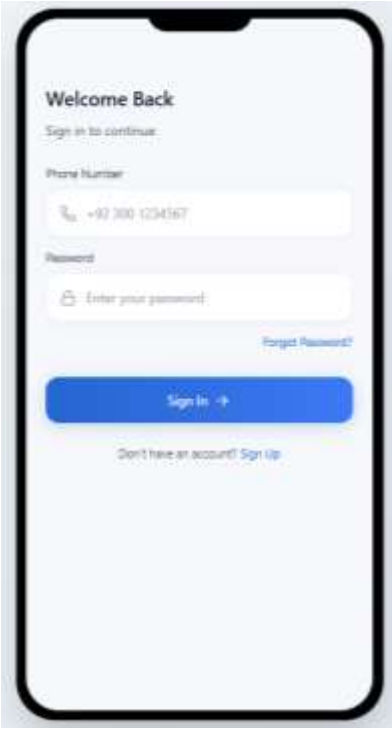
2.3.1. User Interfaces

The system includes:

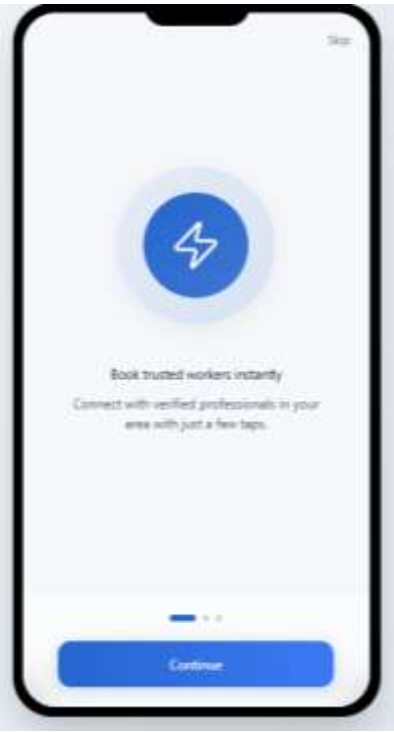
- **Customer App UI:**
  - Home screen with categories
  - Search bar
  - Technician profile pages
  - Booking form
  - Real-time tracking map
  - Payment screen
- **Service Provider App UI:**
  - Job list
  - Availability scheduling
  - Earnings dashboard
- **Admin Dashboard:**
  - Web interface with tables, charts, CRUD operations

UI will follow standard Material Design & iOS UI conventions.

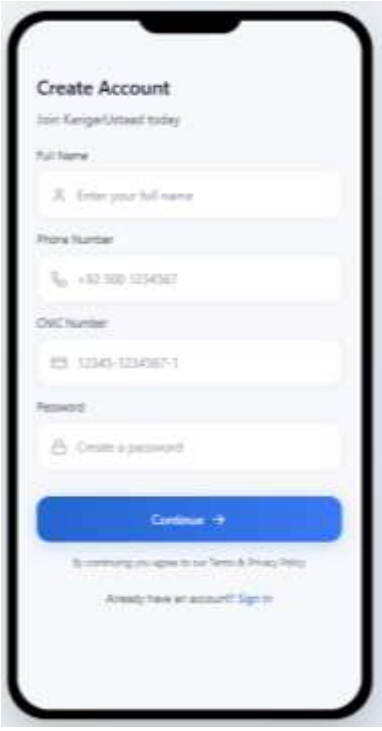
SCREEN 1 SPLASH



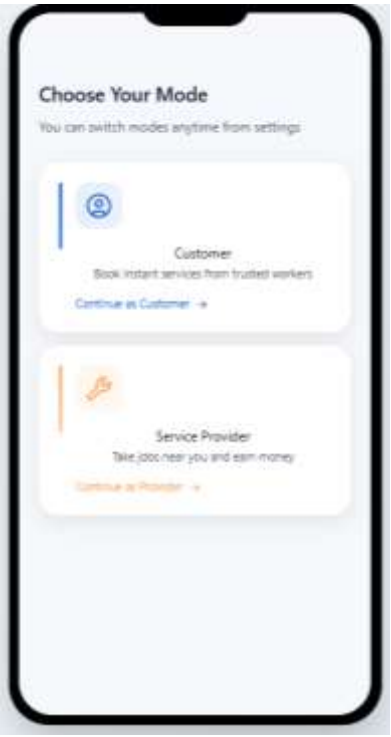
SCREEN 3 SIGNIN



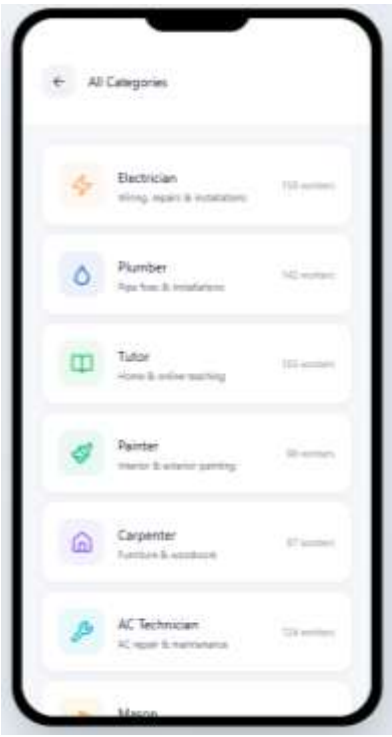
SCREEN 2 LOADING



SCREEN 4 SIGNUP

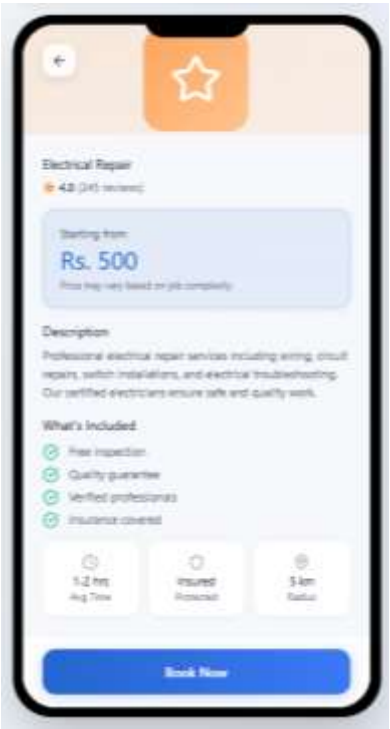


SCREEN 5 MODE SELECTION

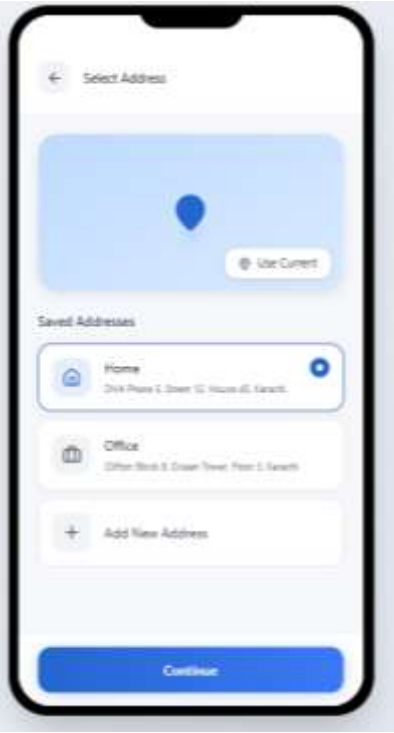


SCREEN 7 CATEGORIES

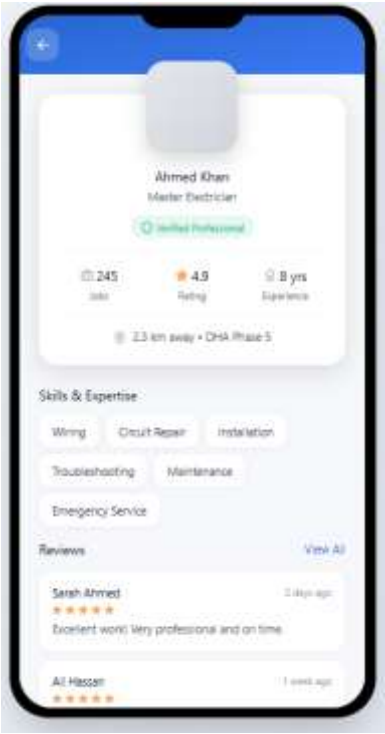
SCREEN 6 DASHBOARD



SCREEN 8 BOOKING

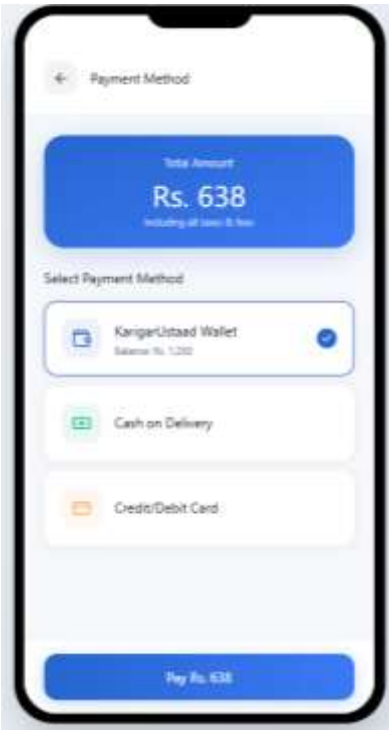


SCREEN 9 BOOKING CONFIRMATION



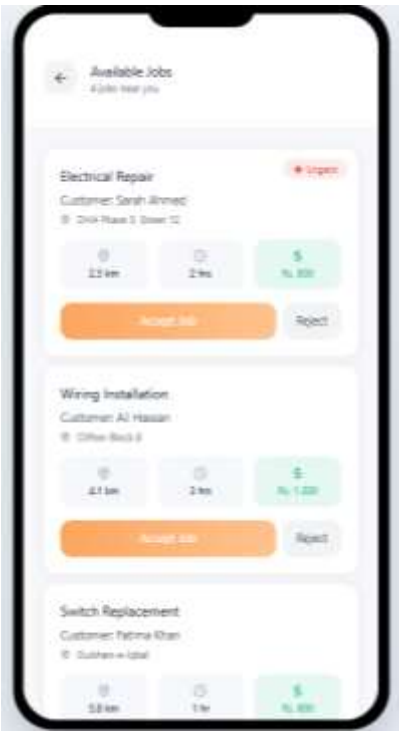
SCREEN 11 WORKER PROFILE

SCREEN 10 WORKER DETAILS

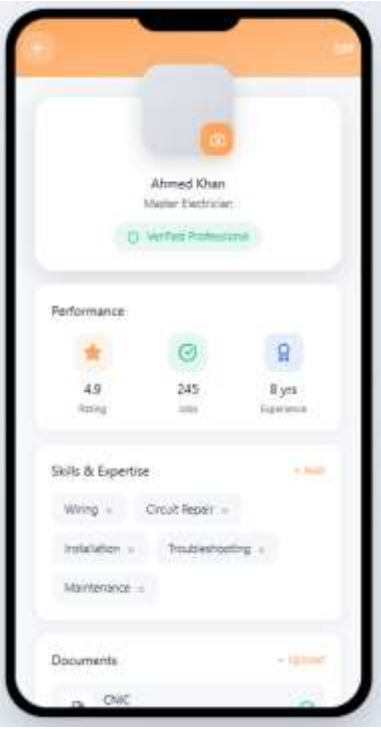


SCREEN 12 PAYMENT

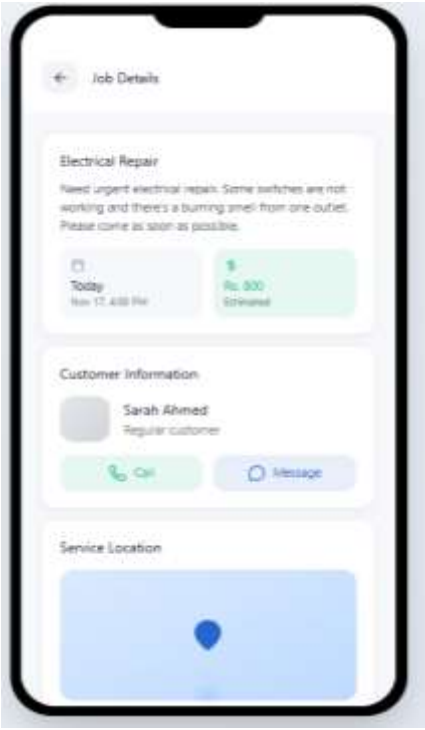
SCREEN 13 WORKER DASHBOARD



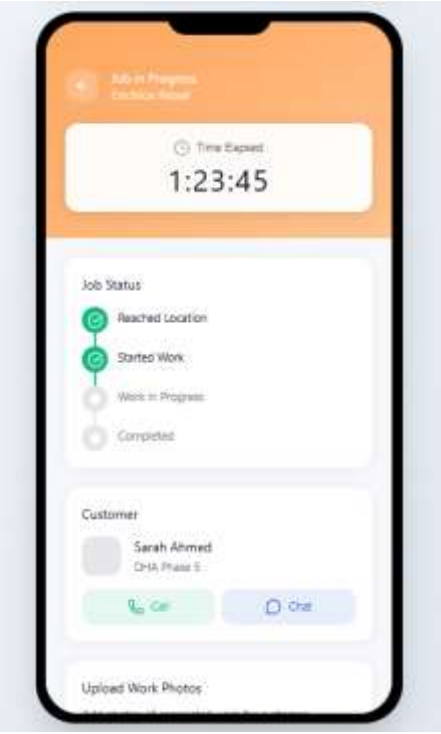
SCREEN 15 JOB REQUESTS



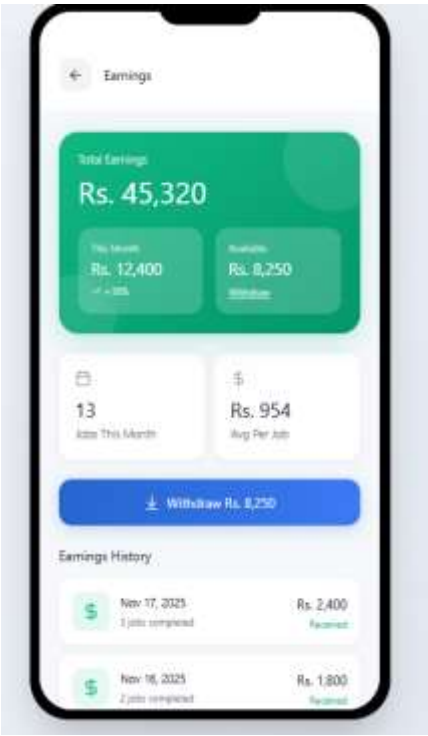
SCREEN 14 PROFILE



SCREEN 16 JOB DETAILS



SCREEN 17 PROGRESS TRACKING



SCREEN 18 EARNINGS

2.3.2. Hardware Interfaces

- Mobile devices (Android/iOS)
- GPS hardware
- Camera for KYC verification
- Payment device integration through mobile internet

2.3.3. Software Interfaces

- RESTful API for communication between apps and server
- Google Firebase for notifications
- Google Maps for navigation
- Third-party payment gateway APIs
- Internal AI microservice APIs

2.3.4. Communications Interfaces

- HTTPS communication
- JSON-based API request/response
- WebSocket for real-time tracking
- SMS/email for verification and notifications
- Data encryption using TLS 1.2+

2.4. System Features

2.4.1. User Registration & Authentication

2.4.1.1. Description and Priority

Allows customers and service providers to create accounts, log in, and authenticate securely.

Priority: High

2.4.1.2. Stimulus/Response Sequences

User enters phone or email → System sends OTP

User submits OTP → System verifies

Successful login → Redirect to dashboard

2.4.1.3. Functional Requirements

REQ-UR-1: System shall allow users to register using phone, email, or social login.

REQ-UR-2: System shall send OTP for verification.

REQ-UR-3: System shall encrypt and store user credentials securely.

REQ-UR-4: System shall lock the account after 5 failed attempts.

2.4.2. Service Booking & Scheduling

2.4.2.1. Description and Priority

Users can book professionals, choose date/time, and track status.

Priority: High

2.4.2.2. Stimulus/Response Sequences

User selects service category

User views available providers

User selects provider → confirms booking

System assigns and notifies provider

Provider accepts → Status updates

2.4.2.3. Functional Requirements

REQ-SB-1: System shall allow users to browse service categories.

REQ-SB-2: System shall provide real-time availability of providers.

REQ-SB-3: System shall show estimated service cost.

REQ-SB-4: System shall notify users of booking confirmation.

2.4.3. AI-Powered Matching & Dynamic Pricing

2.4.3.1. Description and Priority

AI engine recommends best service provider based on skills, proximity, and ratings.

Priority: Medium

2.4.3.2. Functional Requirements

REQ-AI-1: System shall analyze provider ratings, skills, and distance.
REQ-AI-2: System shall generate dynamic price estimates.
REQ-AI-3: System shall detect fake reviews.

2.5. Nonfunctional Requirements

2.5.1. Performance Requirements

- Response time must be < 3 seconds.
- System must support 10,000+ concurrent users.

2.5.2. Safety Requirements

- Must warn users about hazardous tasks and recommend safe practices.
- Technicians must be verified before field deployment.

2.5.3. Security Requirements

- End-to-end encrypted communication
- Role-based access control
- Mandatory KYC for providers

2.5.4. Usability Requirements

- App UI must be simple and accessible for low-literacy users
- Local language support (Urdu)

2.5.5. Reliability Requirements

- System uptime ≥ 99%
- Auto-retry mechanisms for failed API calls

2.5.6. Maintainability/Supportability Requirements

- Must follow modular microservice architecture
- Code must comply with company coding standards

2.5.7. Portability Requirements

- App should run on multiple OS versions
- Backend deployable on any cloud platform

2.5.8. Efficiency Requirements

- AI modules must run with < 40% CPU load
- App memory usage < 200 MB

2.6. Domain Requirements

- Must comply with Pakistani consumer protection regulations
- Must log all transactions for auditing
- Localization: Urdu and English support
- Payment receipts must follow SECP digital invoicing guidelines

Chapter 3

Use Case Analysis

Chapter 3: Use Case Analysis

Use case analysis provides a structured understanding of how users interact with the system and how the system responds to those interactions. This chapter outlines the functional behavior of the system by identifying key actors, their goals, and the major services provided by the application. It also ensures that all system requirements described earlier are mapped into user-oriented scenarios, helping developers, testers, and stakeholders clearly visualize the expected system operations. Through the use case model and detailed use case descriptions, this chapter forms the foundation for system design and implementation.

3.1. Use Case Model

The use case model represents a high-level overview of all interactions between the users (actors) and the system. It identifies the primary actors, their roles, and the major functionalities they access. This model serves as a blueprint for understanding system behavior from the user's perspective and highlights the boundaries of the system. By illustrating user goals and the system’s response, the use case model ensures that each function aligns with the project objectives and supports the overall workflow.

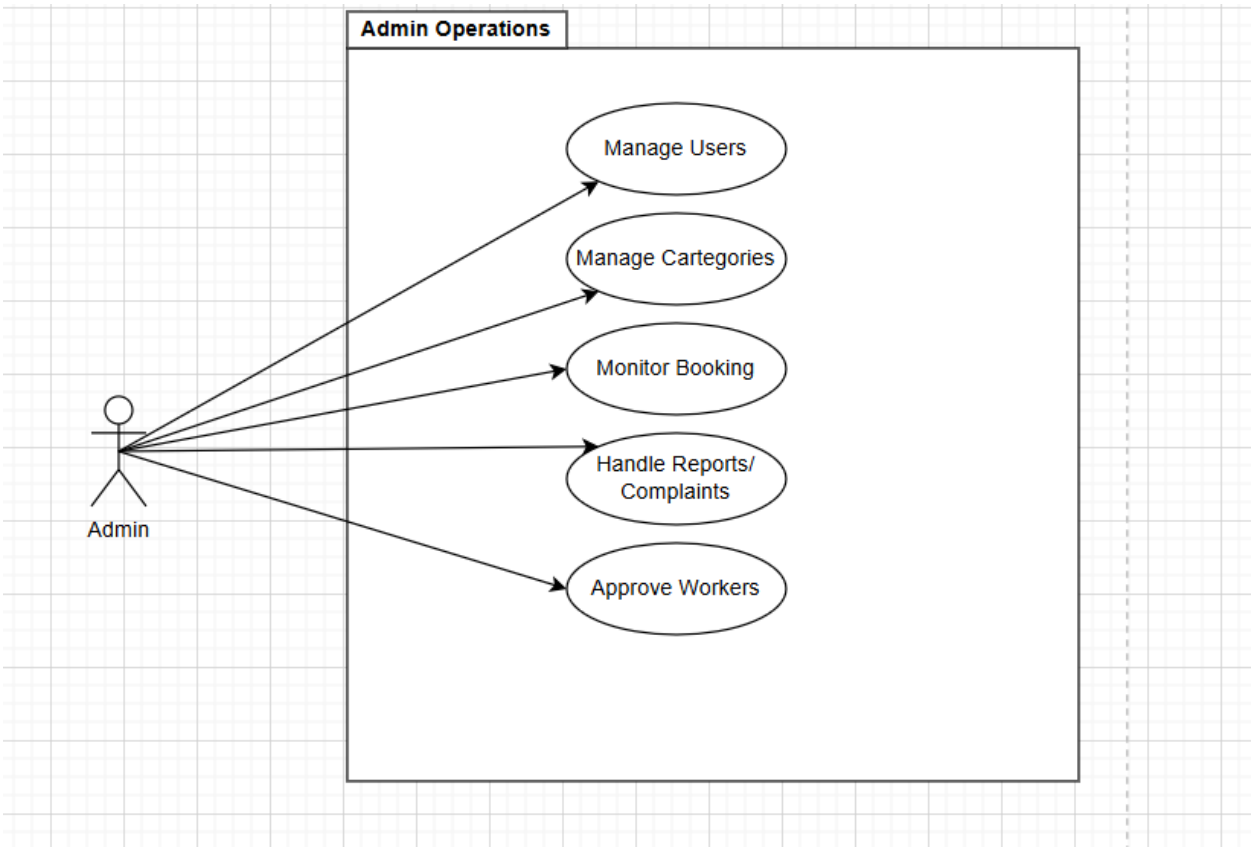


Figure 3 (ADMIN) Use Case Model

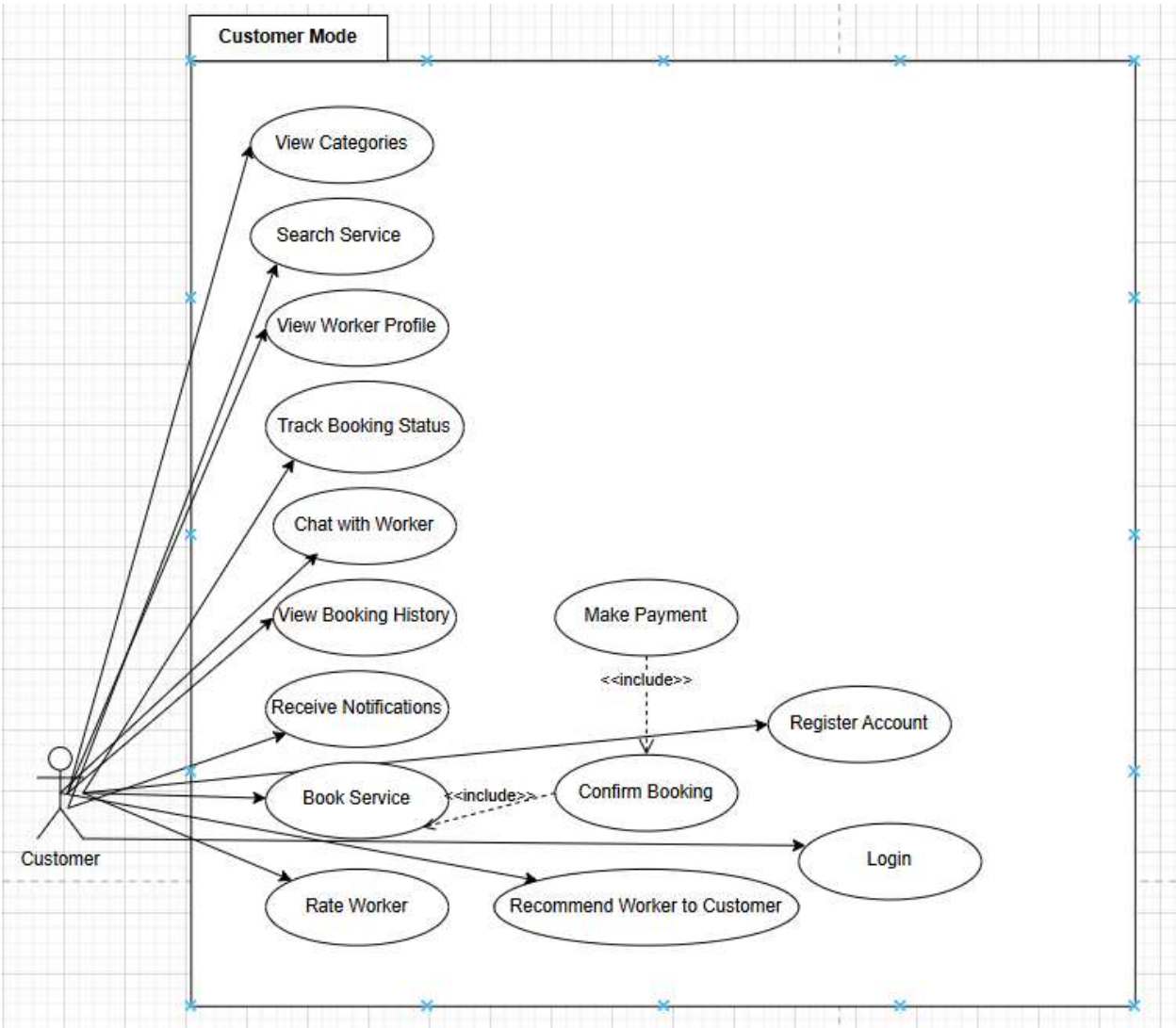


Figure 4 (Customer) Use Case Model

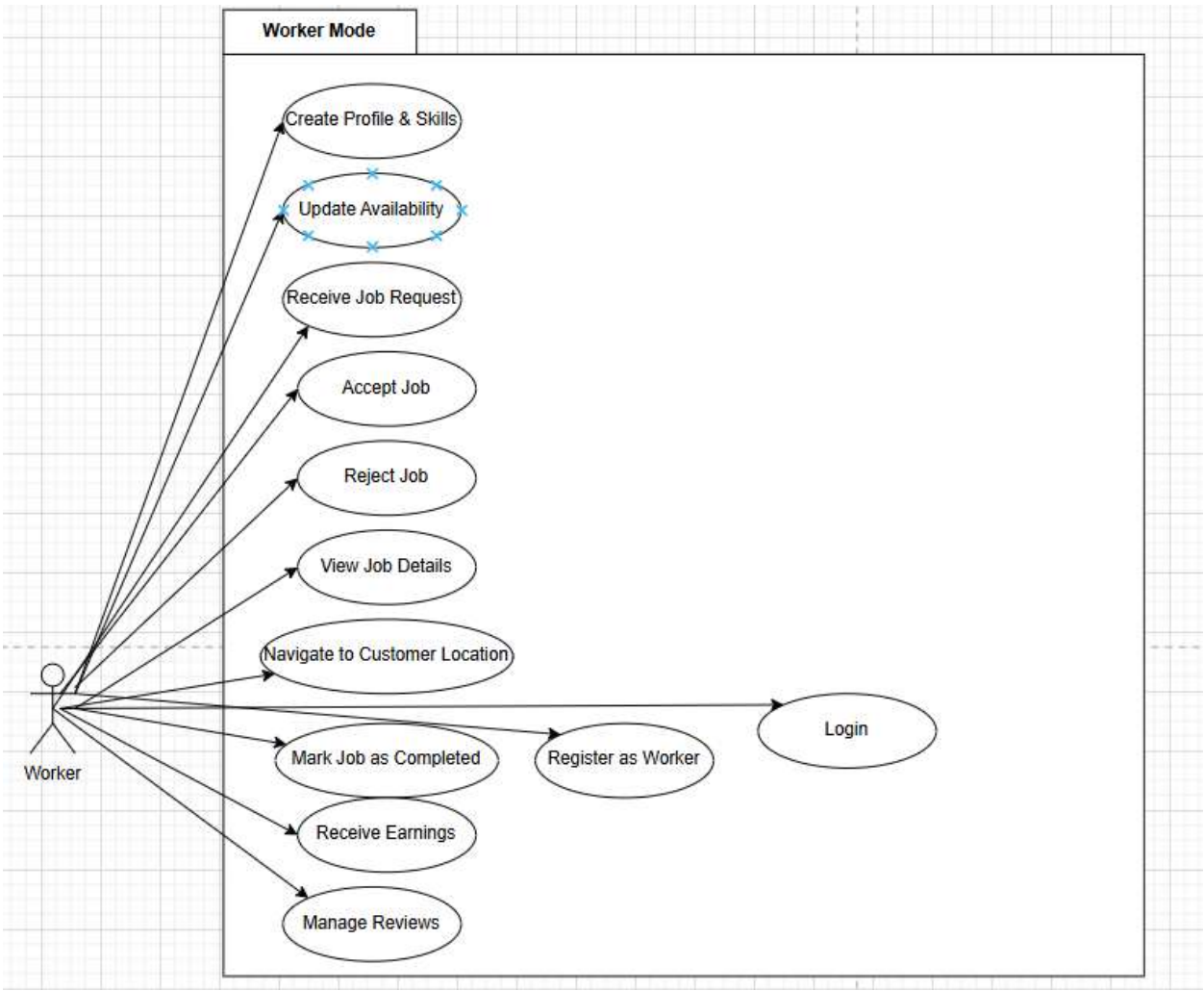


Figure 5 (worker) Use Case Model

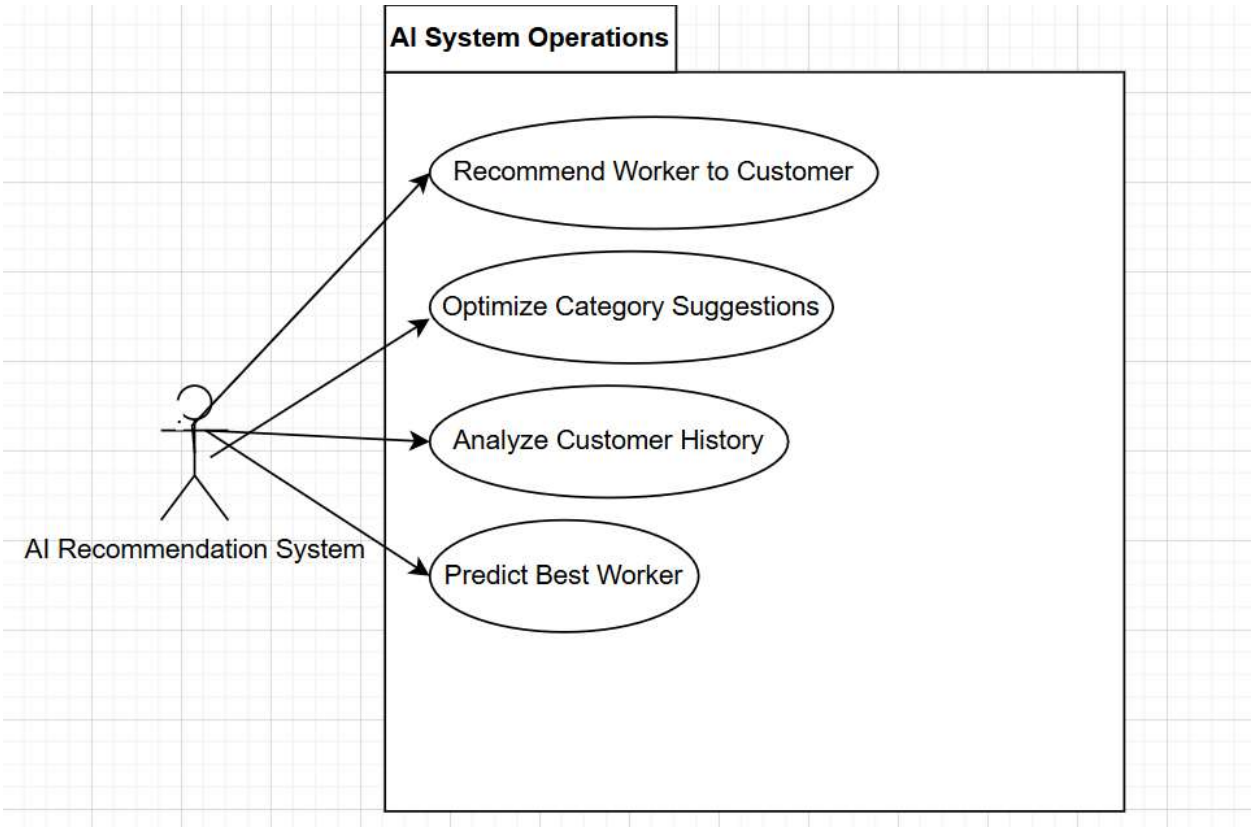


Figure 6 (AI-SYSTEM) Use Case Model

3.2. Use Cases Description

This section provides detailed descriptions of each use case identified in the model. Each use case includes essential elements such as the use case name, actors involved, preconditions, postconditions, triggers, and step-by-step flows. These descriptions help clarify how the system should behave during specific interactions and guide both development and testing teams. By breaking down each process into structured scenarios, the use case descriptions contribute to a clear, unambiguous, and verifiable understanding of system functionality.

Use Case 1: User Registration & Authentication

Table 3 Use Case 1: User Registration & Authentication

Use Case ID	UC-01
Use Case Name	User Registration & Authentication
Actor(s)	Customer, Service Provider
Preconditions	User has a mobile device and internet access.
Postconditions	User account is created, verified, and stored in the system.

Use Case ID	UC-01
Trigger	User taps "Register" or "Login" button.

Main Flow

1. User enters phone for registration.
2. System sends OTP to provided contact.
3. User enters OTP.
4. System verifies OTP.
5. System creates a new user account.
6. System redirects user to the dashboard.

Alternate Flow

- **OTP Invalid:** System displays error → User re-enters OTP.
- **User Already Exists:** System prompts for login instead of registration.

Exceptions

- No internet connection
- SMS/Email not delivered

Use Case 2: Browse Services & View Providers

Table 4 Use Case 2: Browse Services & View Providers

Use Case ID	UC-02
Use Case Name	Browse Services & View Providers
Actor(s)	Customer
Preconditions	User is logged in.
Postconditions	User views available providers for a selected service.
Trigger	User selects a service category.

Main Flow

1. User opens home screen.
2. User selects a service category (e.g., Electrician).
3. System fetches providers.
4. System displays provider details (ratings, price, availability).

Alternate Flow

- No provider available → System shows “No providers available” message.

Exceptions

- Backend server timeout

Use Case 3: Book a Service

Table 5 Use Case 3: Book a Service

Use Case ID	UC-03
Use Case Name	Service Booking & Scheduling
Actor(s)	Customer
Preconditions	User selected a service provider. User is logged in.
Postconditions	A booking is created and stored. Provider is notified.
Trigger	User taps “Book Now.”

Main Flow

1. User selects provider.
2. User selects date and time.
3. User confirms booking.
4. System generates a booking ID.
5. System sends notification to provider.
6. System updates booking status to "Pending Acceptance."

Alternate Flow

- Provider rejects → User gets notified & chooses another provider.
- User cancels before acceptance → Booking is deleted.

Exceptions

- Payment gateway unavailable
- Provider unavailable unexpectedly

Use Case 4: Provider Accepts/Rejects Booking

Table 6 Use Case 4: Provider Accepts/Rejects Booking

Use Case ID	UC-04
Use Case Name	Provider Booking Response
Actor(s)	Service Provider
Preconditions	Provider is online and has a valid account.

Use Case ID	UC-04
Postconditions	Booking status changes to Accepted/Rejected.
Trigger	Provider receives booking notification.

Main Flow

1. Provider receives booking alert.
2. Provider views booking details.
3. Provider accepts booking.
4. System updates status to "Accepted."
5. Customer receives confirmation.

Alternate Flow

- Provider rejects → System assigns next available provider.

Exceptions

- Provider app not responding
- Provider loses internet connectivity

Use Case 5: Real-Time Tracking

Table 7 Use Case 5: Real-Time Tracking

Use Case ID	UC-05
Use Case Name	Real-Time Tracking of Provider
Actor(s)	Customer, Service Provider
Preconditions	Provider has accepted the booking. GPS enabled.
Postconditions	Customer views live provider location.
Trigger	Customer opens "Track Provider" screen.

Main Flow

1. Provider starts job navigation in app.
2. System retrieves GPS data.
3. System updates customer screen every X seconds.
4. Customer sees provider’s route & ETA.

Alternate Flow

- Provider disables GPS → System shows “Location unavailable.”

Exceptions

- GPS inaccuracy
- Slow internet connection

Use Case 6: Payment Processing

Table 8 Use Case 6: Payment Processing

Use Case ID	UC-06
Use Case Name	Payment & Invoice Generation
Actor(s)	Customer, Payment Gateway
Preconditions	Service completed. User has payment method.
Postconditions	Payment processed and invoice stored.
Trigger	User taps “Proceed to Payment.”

Main Flow

1. User selects payment method (JazzCash/EasyPaisa/Card).
2. System sends payment request to gateway.
3. Gateway confirms payment.
4. System updates booking as "Paid."
5. System generates digital invoice.

Alternate Flow

- Payment failed → User retries or selects another method.

Exceptions

- Gateway downtime
- Insufficient balance

Use Case 7: Rating & Review

Table 9 Use Case 7: Rating & Review

Use Case ID	UC-07
Use Case Name	Service Rating & Feedback

Use Case ID	UC-07
Actor(s)	Customer
Preconditions	Service completed and paid.
Postconditions	Review saved and contributes to provider score.
Trigger	User receives “Rate Your Service” notification.

Main Flow

1. User opens completed booking.
2. User gives star rating.
3. User optionally enters a review.
4. System saves rating and updates provider profile.

Alternate Flow

- User skips → System sends reminder later.

Exceptions

- Review submission fails due to server error

Use Case 8: Provider Profile Management

Table 10 Use Case 8: Provider Profile Management

Use Case ID	UC-08
Use Case Name	Manage Provider Profile
Actor(s)	Service Provider
Preconditions	Provider is registered and verified.
Postconditions	Updated profile saved in the system.
Trigger	Provider taps “Edit Profile.”

Main Flow

1. Provider updates personal info, skills, and availability.
2. Provider uploads documents for verification.
3. System validates data.
4. System updates provider profile.

Alternate Flow

- Missing required documents → System displays error.

Exceptions

- Invalid image upload
- File size too large

Chapter 4

System Design

Chapter 4: System Design

System design defines how the proposed solution will be structured, implemented, and integrated to meet the functional and non-functional requirements of the KarigarUstaad platform. This chapter provides detailed architectural, structural, and behavioral models that illustrate the internal organization and interactions of the system. These models assist developers and stakeholders in understanding how the system components work together, how data flows through the system, and how different modules interact to deliver required functionality. The diagrams presented in this chapter ensure clarity, consistency, and proper alignment between requirements and implementation.

4.1. Architecture Diagram

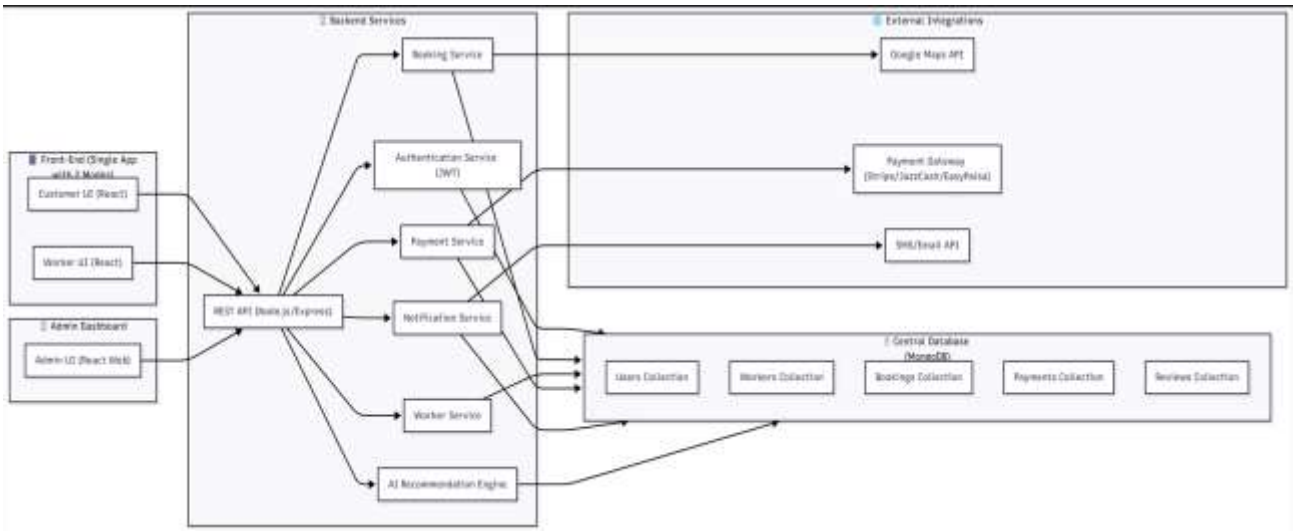


Figure 7 Architecture Diagram

4.2. Domain Model

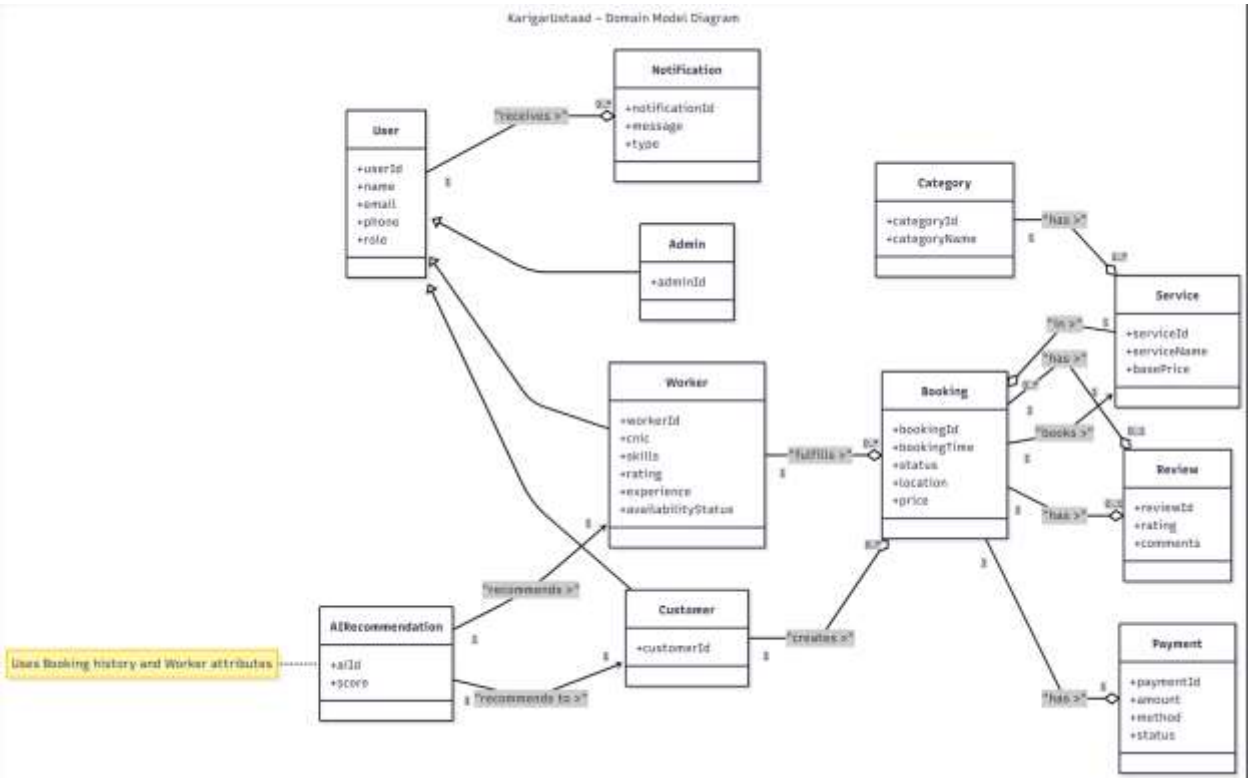


Figure 8 Domain Model

4.3. Entity Relationship Diagram with data dictionary

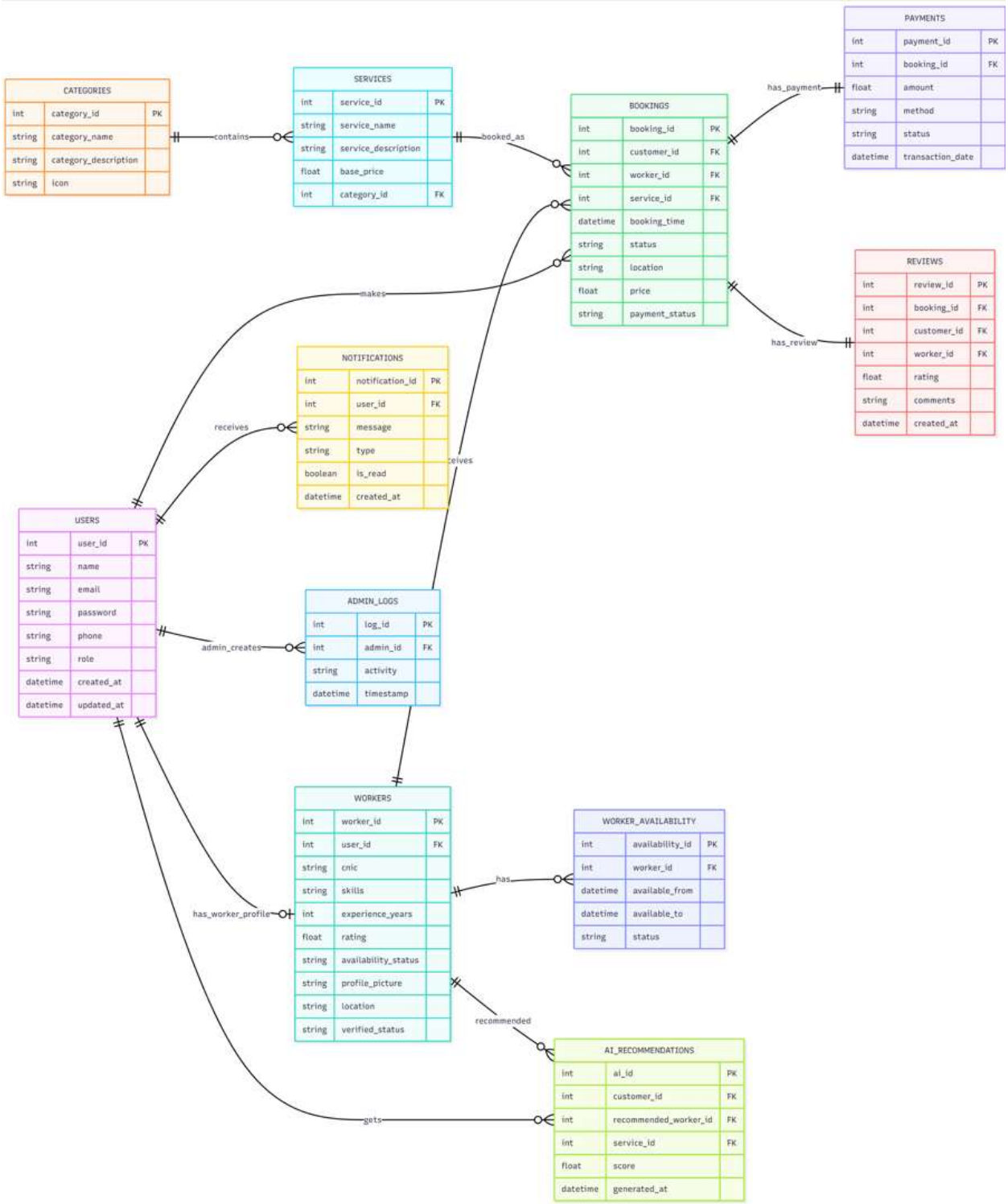


Figure 9 Entity Relationship Diagram

4.4. Class Diagram

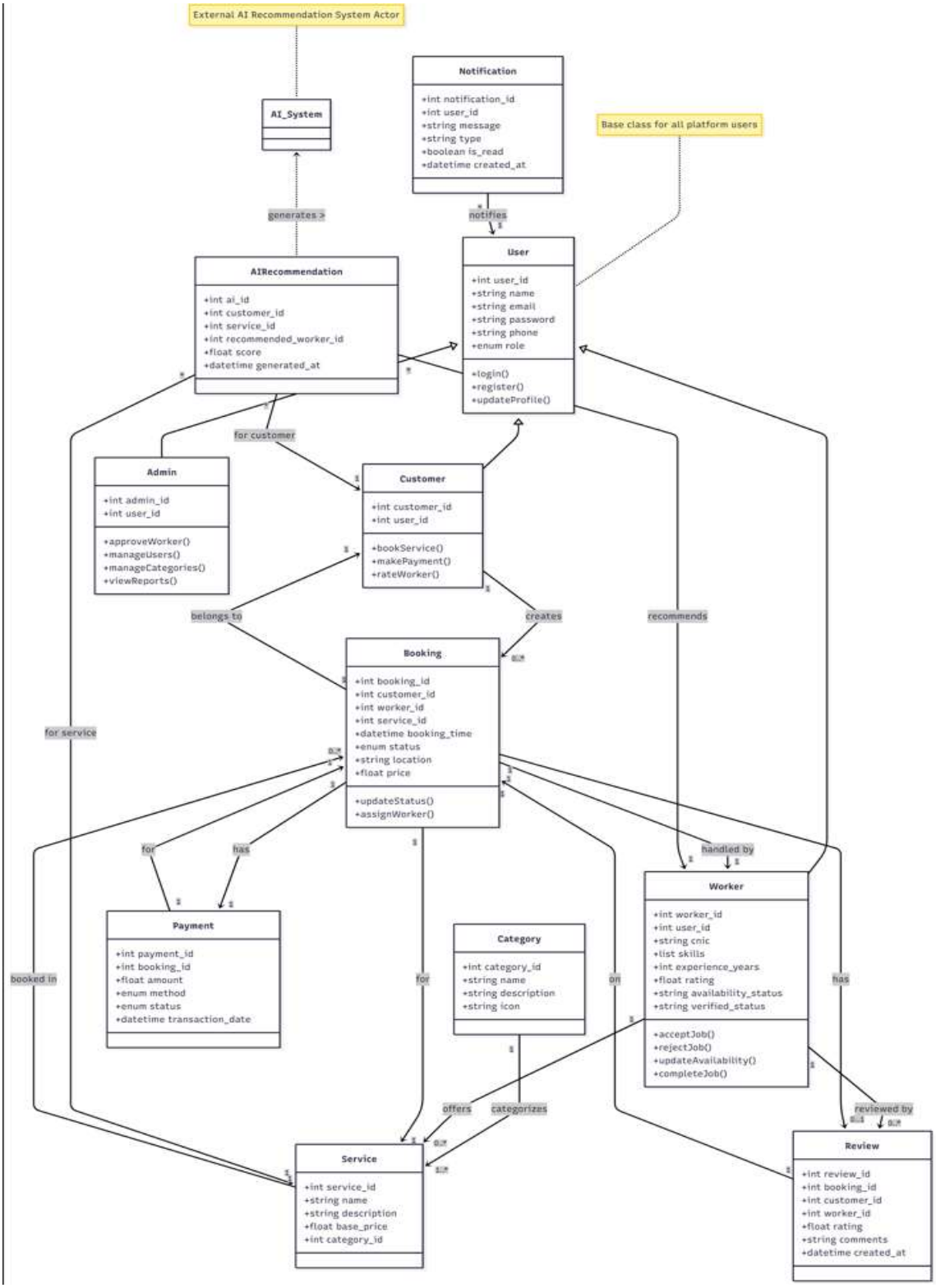


Figure 10 Class Diagram

4.5. Sequence / Collaboration Diagram

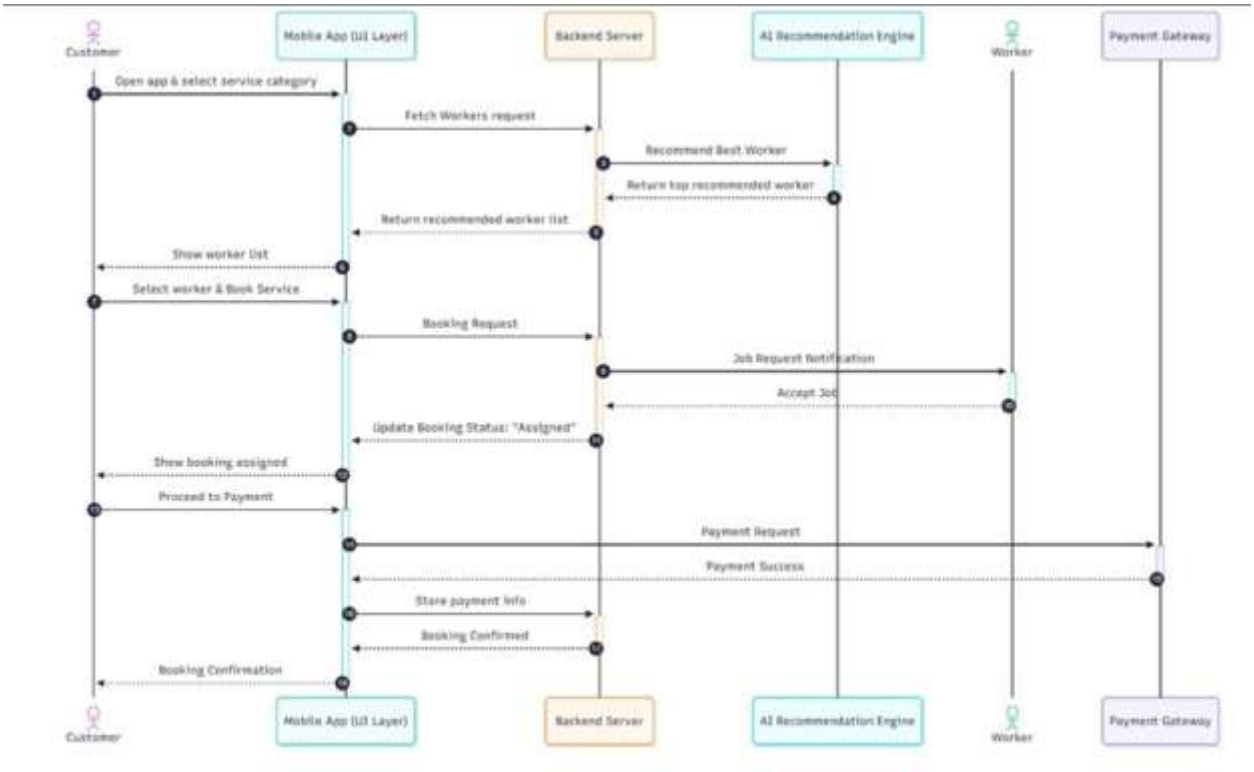


Figure 11 Sequence Diagram

4.6. Operation contracts

Customer Operations

1. **RegisterCustomer**

Purpose: Create a new customer account in the system.

Actors: Customer

Inputs: name, email, phone, password

Outputs: Customer object

Preconditions: Email or phone number is not already registered.

Postconditions: - New Customer object created. - customer.status = “Active”. - customer.registrationDate set to current timestamp.

Main Success Scenario: Customer enters valid data → System validates → New Customer saved.

Failure Scenarios: Duplicate phone/email; weak password; invalid phone format.

2. **LoginCustomer**

Purpose: Authenticate a customer for system access.

Actors: Customer

Inputs: email/phone, password

Outputs: AuthToken

Preconditions: Customer exists and status != “Suspended”.

Postconditions: - Authentication token created. - customer.lastLogin updated.

Main Success Scenario: Customer submits correct credentials → system authenticates → login granted.

Failure Scenarios: Wrong password; suspended account; unregistered email.

3. **BrowseCategories**

Purpose: Display all service categories.

Actors: Customer

Inputs: None

Outputs: List

Preconditions: None

Postconditions: None

Main Success Scenario: Categories fetched and shown.

Failure Scenarios: None.

4. **SearchService**

Purpose: Allow customers to search workers or services.

Actors: Customer

Inputs: searchQuery, filters (rating, price, location)

Outputs: List of matched Workers or Services

Preconditions: None

Postconditions: None

Main Success Scenario: Matching results returned.

Failure Scenarios: No results; invalid filter combinations.

5. **ViewWorkerProfile**

Purpose: Display worker's public profile.

Actors: Customer

Inputs: workerId

Outputs: Worker object

Preconditions: Worker exists and status != "Suspended".

Postconditions: None

Main Success Scenario: Worker profile displayed with reviews, skills, and availability.

Failure Scenarios: Worker does not exist; worker suspended.

Worker Operations

1. **RegisterWorker**

Purpose: Create a worker account.

Actors: Worker

Inputs: name, cnic, skills, email, phone, certificates

Outputs: Worker object

Preconditions: Worker not registered previously.

Postconditions: - worker.status = "PendingApproval". - Worker documents stored.

Main Success Scenario: Worker profile created.

Failure Scenarios: Duplicate CNIC/phone; missing documents.

2. **LoginWorker**

Purpose: Authenticate worker for job handling.

Actors: Worker

Inputs: email/phone, password

Outputs: AuthToken

Preconditions: Worker exists.

Postconditions: Session established; lastLogin updated.

Main Success Scenario: Successful login.

Failure Scenarios: Wrong password; unapproved worker.

3. **UpdateAvailability**

Purpose: Set worker's availability status.

Actors: Worker

Inputs: availability (Available/Unavailable), schedule

Outputs: Updated Worker

Preconditions: Worker logged in.

Postconditions: Availability fields updated.

Main Success Scenario: Availability updated.

Failure Scenarios: Invalid schedule.

4. **ReceiveJobRequest**

Purpose: Notify worker about booking.

Actors: System

Inputs: bookingId

Outputs: Notification

Preconditions: Booking exists.

Postconditions: Worker receives notification.

Main Success Scenario: Notification delivered.

Failure Scenarios: Worker offline.

5. **AcceptJob**

Purpose: Accept a booking request.

Actors: Worker

Inputs: bookingId

Outputs: Updated Booking

Preconditions: booking.status = "Requested".

Postconditions: booking.status = "Accepted". - Notification sent to customer.

Main Success Scenario: Worker accepts successfully.

Failure Scenarios: Job already accepted by someone else.

4.7. Activity Diagram

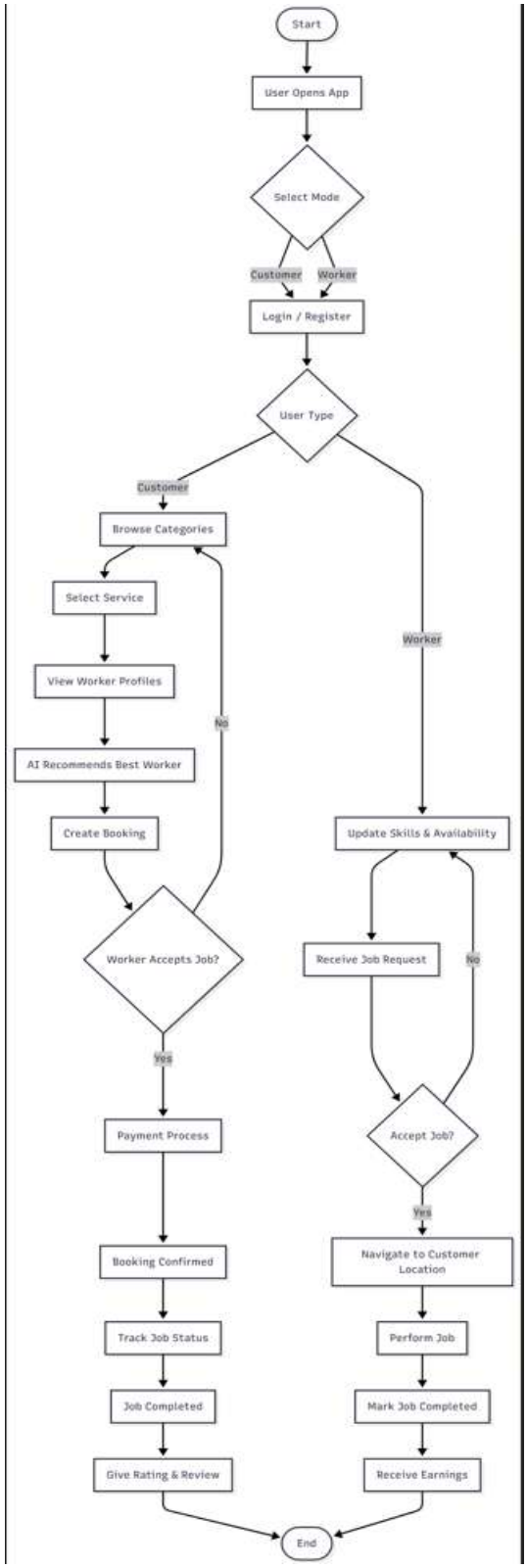


Figure 12 Activity Diagram

4.8. State Transition Diagram

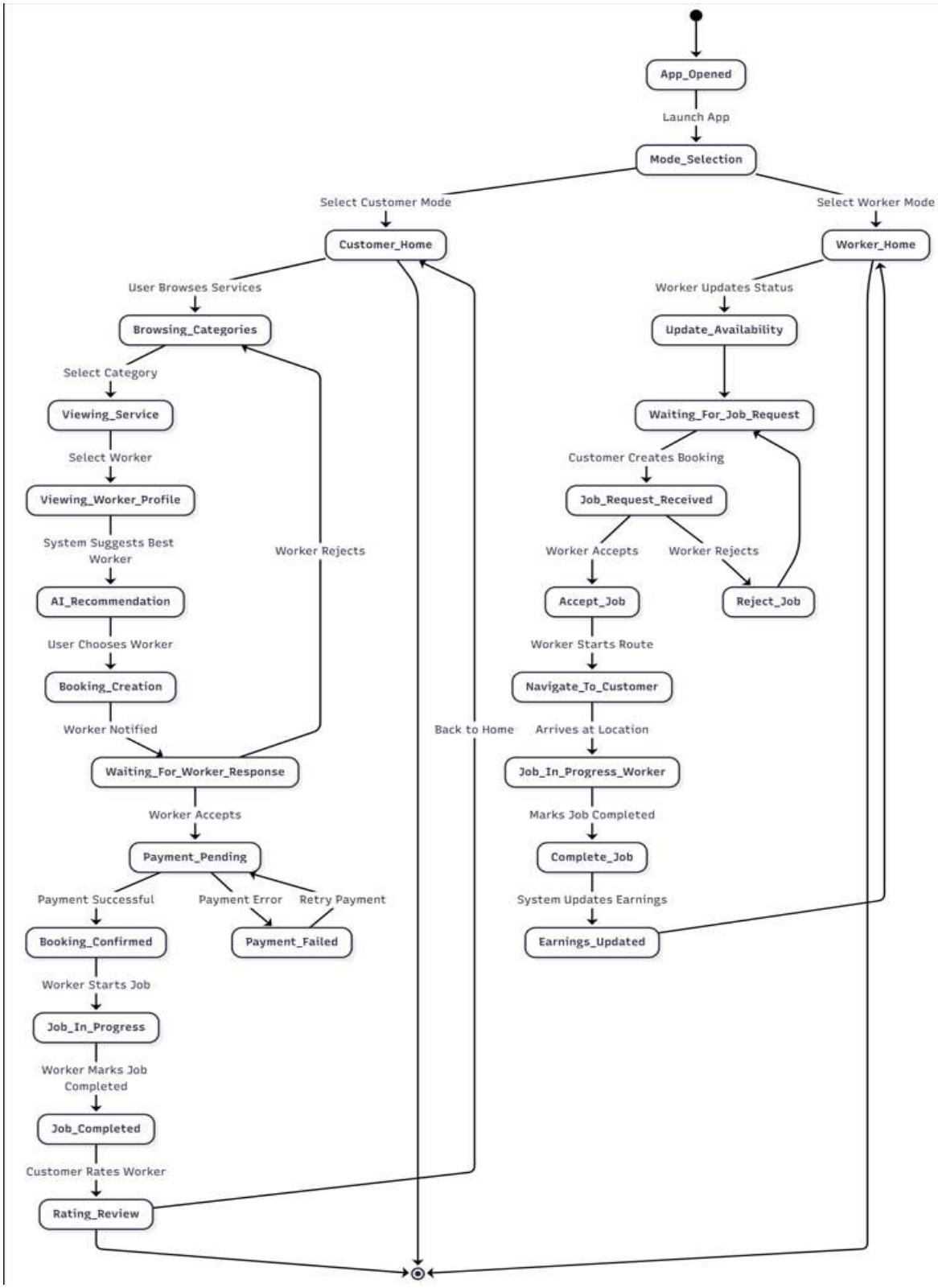


Figure 13 State Transition Diagram

4.9. Component Diagram

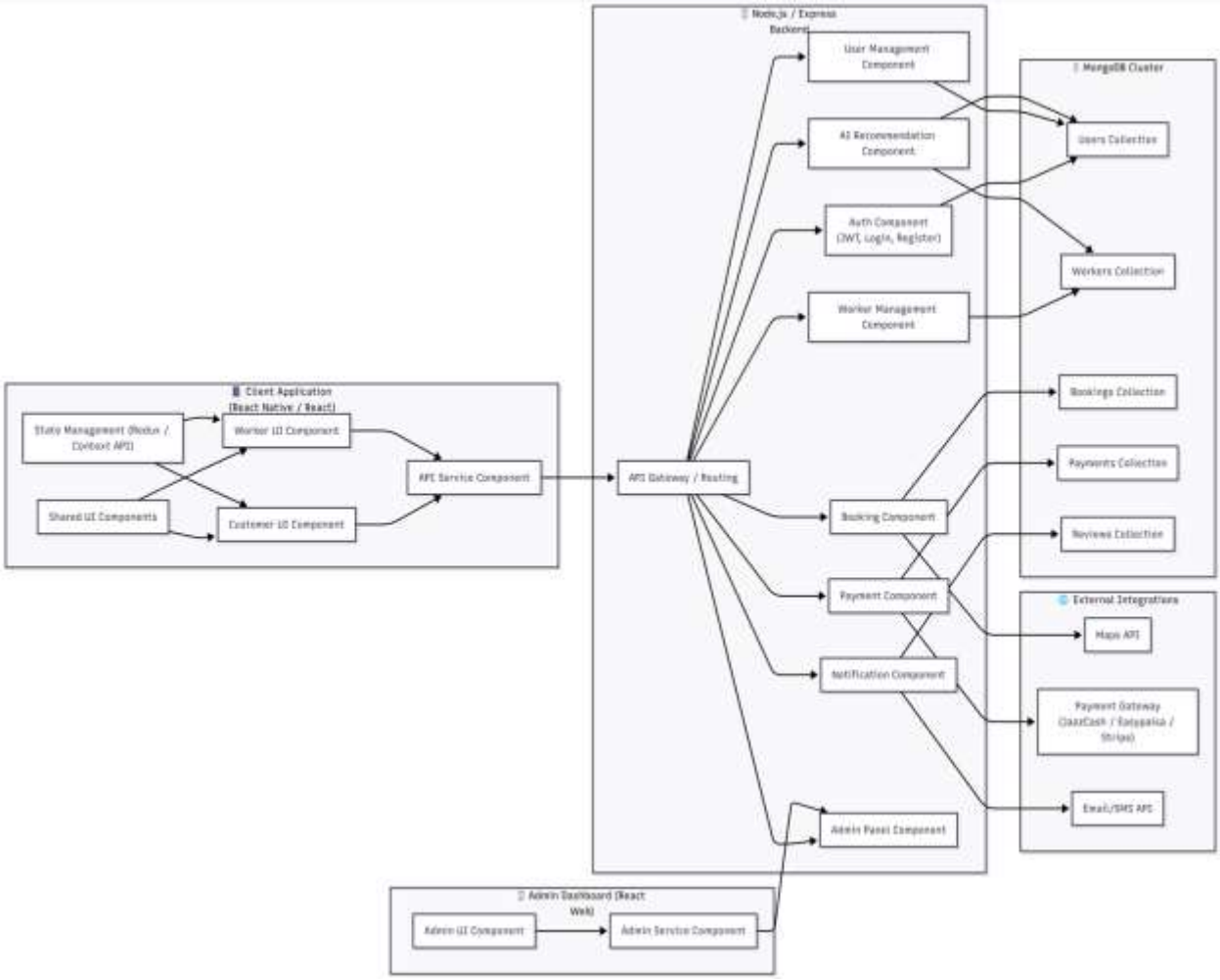


Figure 14 Component Diagram

4.10. Deployment Diagram

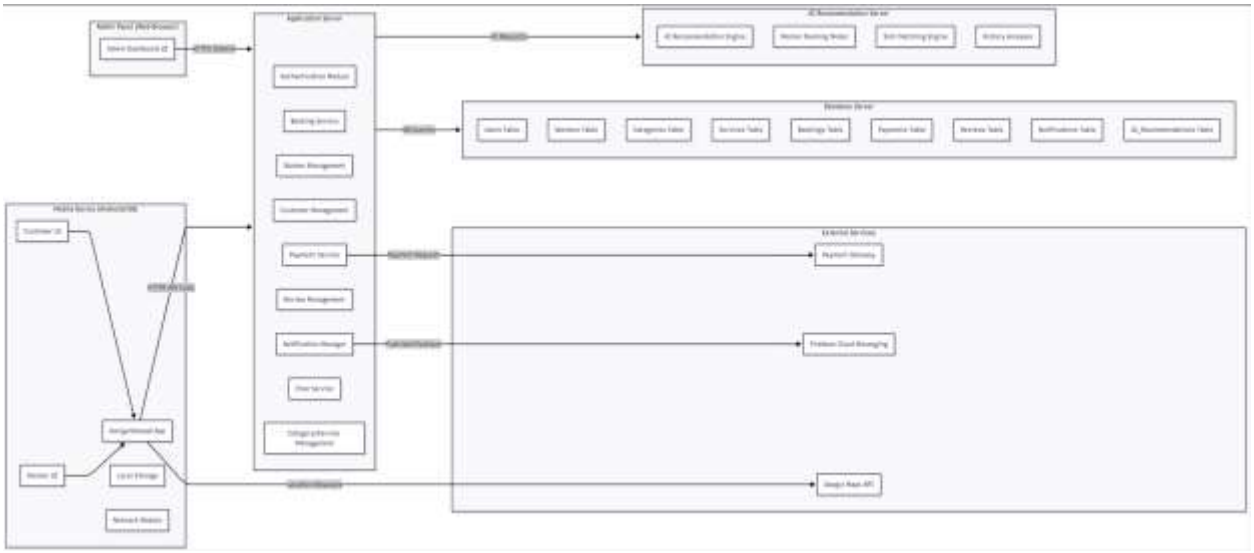


Figure 15 Deployment Diagram

4.11. Data Flow diagram

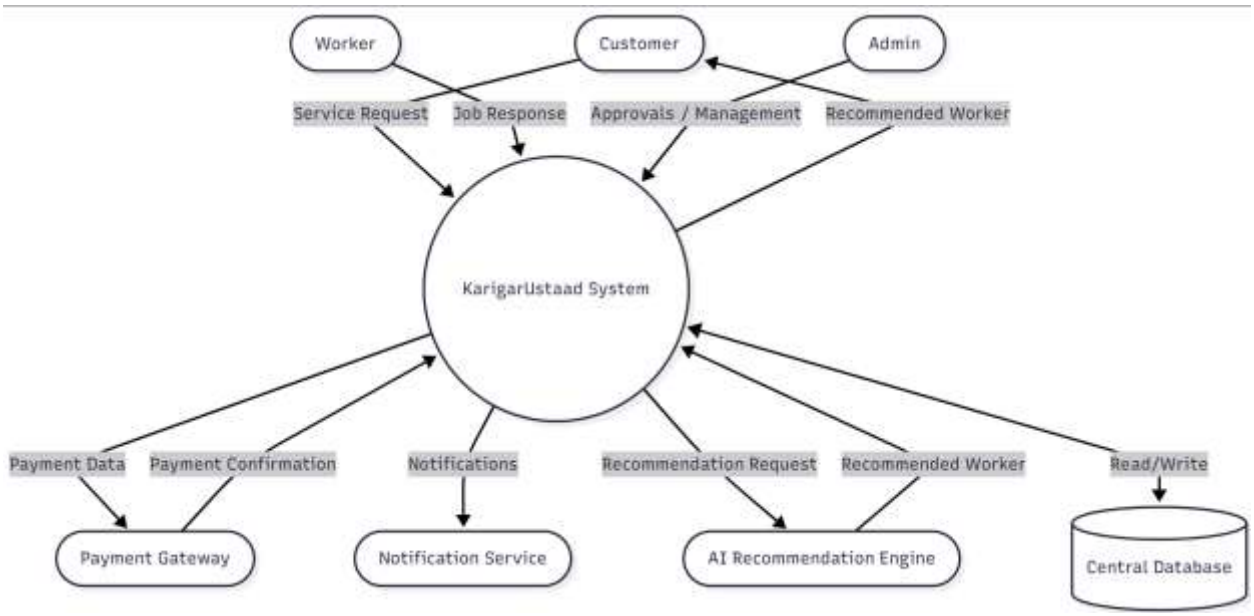


Figure 16 Data Flow Diagram Level 0

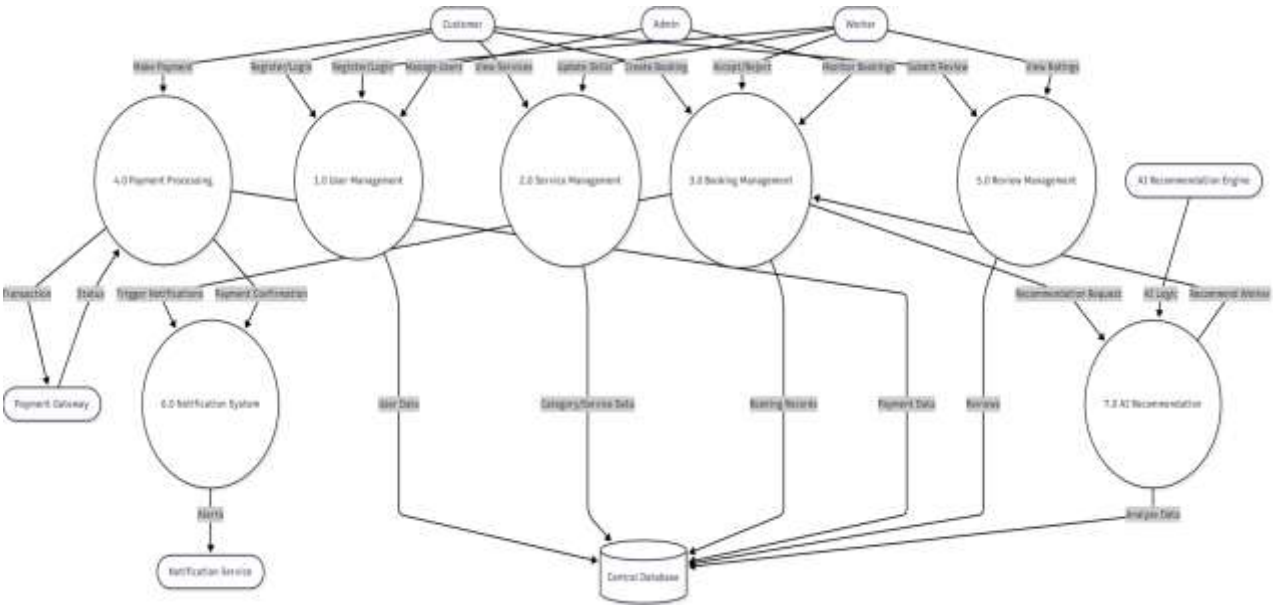


Figure 17 Data Flow Diagram Level 1

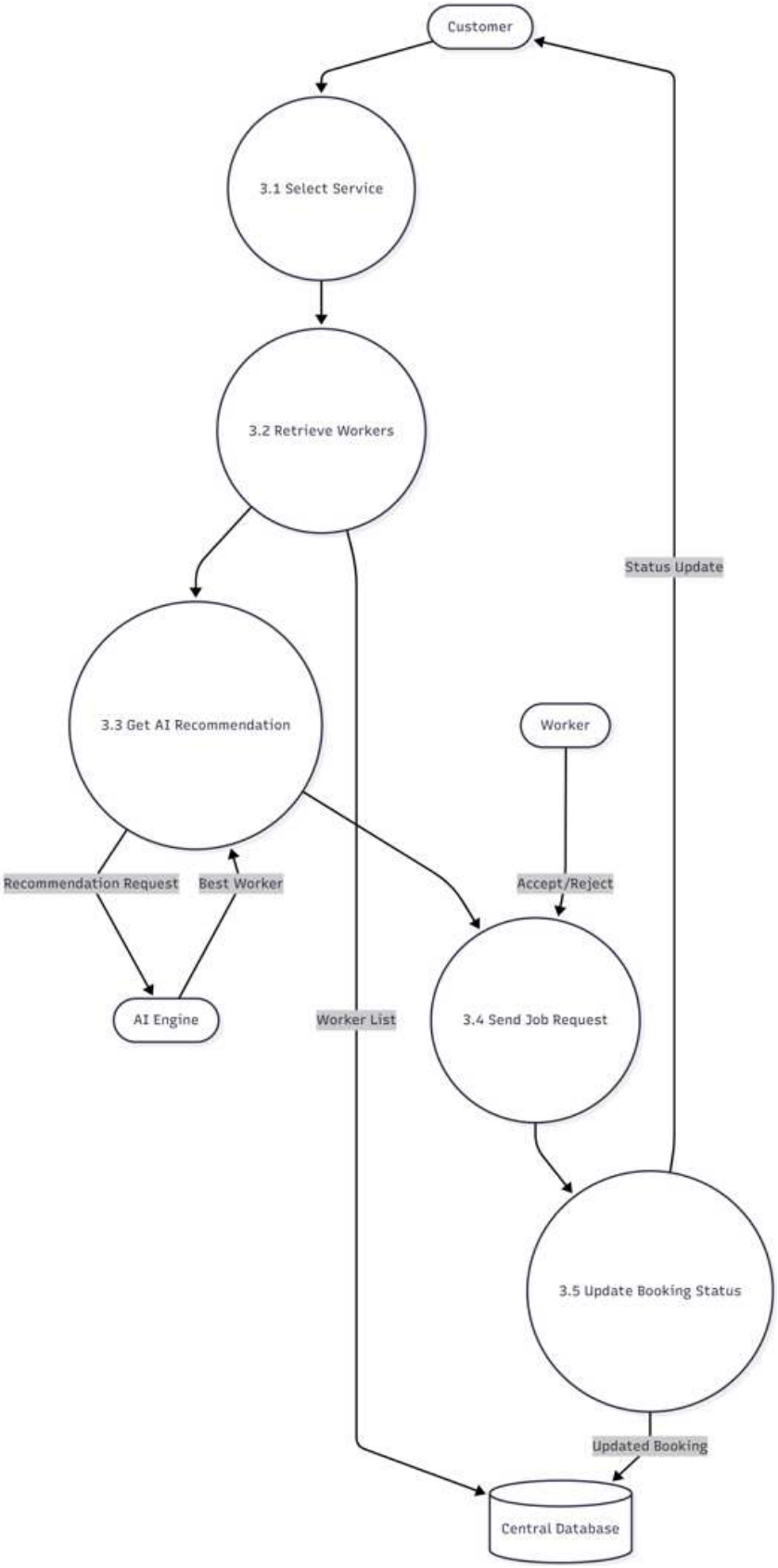


Figure 18 Data Flow Diagram Level 2 (A)

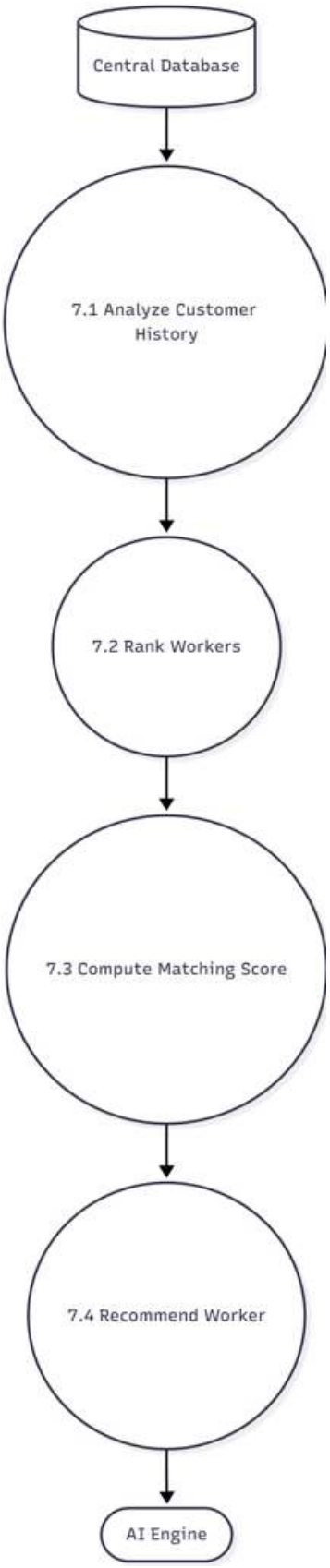


Figure 19 Data Flow Diagram Level 2 (B)

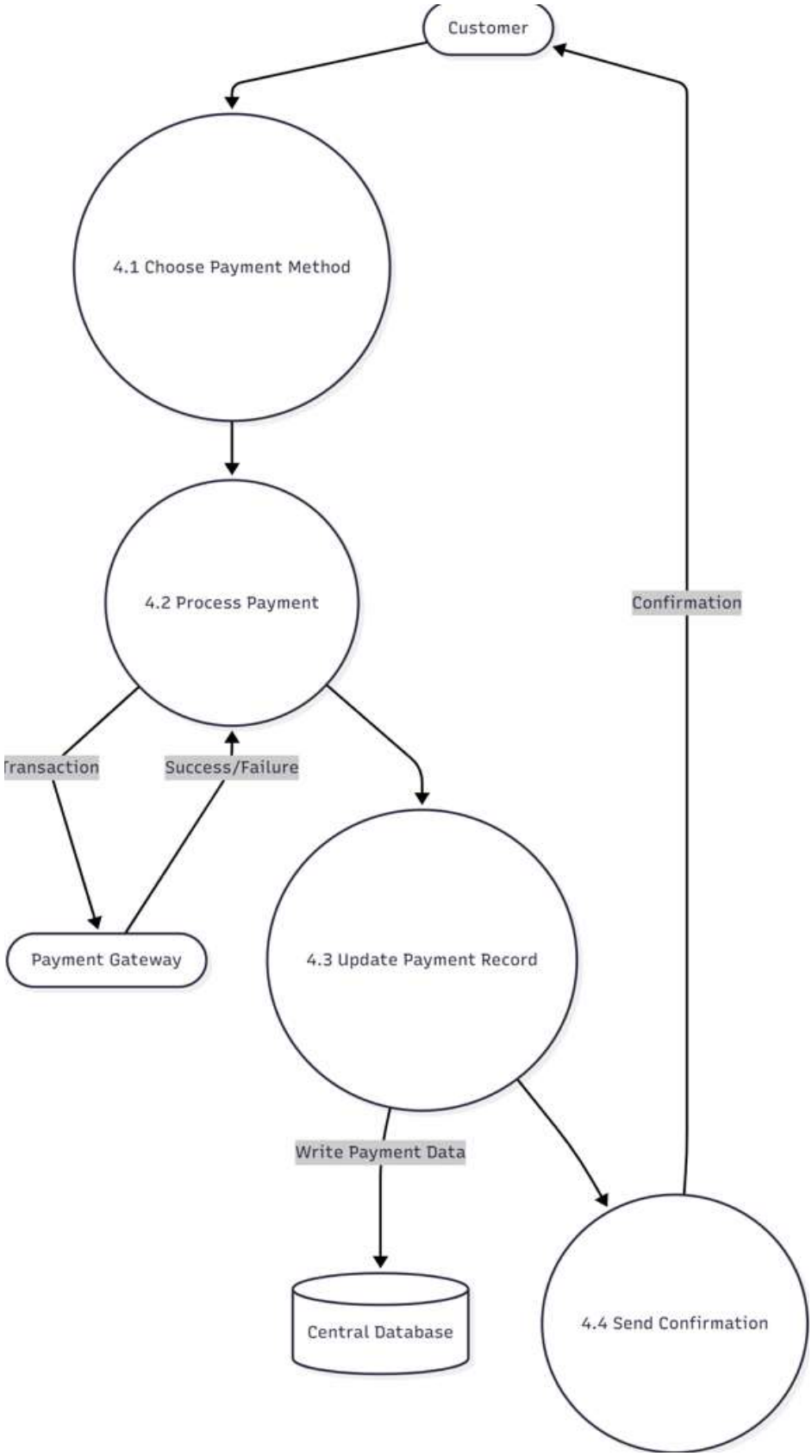


Figure 20 Data Flow Diagram Level 2 (C)

Chapter 5

Implementation

Chapter 5: Implementation

This chapter details the practical implementation of the KarigarUstaad platform using the MERN stack. It explains how system models were converted into working modules, important flow logic, pseudo-code of critical processes, libraries and tools used, deployment setup, coding practices, and version control. The implementation phase transforms the conceptual design into functional components, including the frontend, backend APIs, database logic, and communication between customer and worker modules.

5.1. Important Flow Control/Pseudo codes

Below are the key pseudo-code blocks that represent the primary logic of critical features in KarigarUstaad.

1. User Registration with OTP Verification (MERN)

```
FUNCTION registerUser(phone):
  IF user exists in MongoDB:
    RETURN "User Already Registered"
  otp = generateOTP()
  sendOTP(phone, otp)
  INPUT userEnteredOTP
  IF userEnteredOTP == otp:
    saveUserToMongoDB(phone)
    RETURN "Registration Successful"
  ELSE:
    RETURN "Verification Failed"
```

2. AI Worker Recommendation Logic

```
FUNCTION recommendWorker(category, userLocation):
  workers = MongoDB.find({category})
  FOR each worker:
    score = (worker.rating * 0.4) +
        (worker.completedJobs * 0.3) +
```

```
            (distance(userLocation, worker.location) * 0.3)
        worker.score = score
    RETURN worker WITH highest score
```

3. Booking Flow

```
FUNCTION createBooking(customerId, workerId, schedule):
    IF worker.isAvailable(schedule):
        booking = saveBookingToDB(customerId, workerId, schedule, "Pending")
        notifyWorker(workerId, "New Job Request")
        RETURN booking
    ELSE:
        RETURN "Worker Not Available"
```

4. Worker Accept/Reject Flow

```
FUNCTION workerResponse(bookingId, decision):
    IF decision == "Accept":
        update booking status to "Accepted"
        notifyCustomer("Accepted")
    ELSE:
        update booking status to "Rejected"
        autoAssignAlternativeWorker()
```

5. Secure Payment Flow

```
FUNCTION payment(bookingId, cardDetails):
    gatewayResponse = paymentGateway.charge(cardDetails)
    IF gatewayResponse.success:
        updateBookingStatus("Paid")
        saveTransaction()
        RETURN "Payment Successful"
    ELSE:
        RETURN "Payment Failed"
```

5.2. Components, Libraries, Web Services and stubs

Frontend (React.js)

- React Components (Screens, Cards, Buttons)
- Tailwind CSS for styling
- Axios for API calls
- React Router for navigation
- Context API / Redux for state management

Backend (Node.js + Express.js)

- Express Router
- Middleware (Auth, Error Handling, Logging)
- JSON Web Tokens (JWT)
- Bcrypt.js for hashing
- Nodemailer / SMS Gateway for OTP
- Multer for image uploads

Database (MongoDB + Mongoose)

- Collections: Users, Workers, Bookings, Payments, Reviews
- Mongoose schemas and model validation
- GeoJSON for worker location

External Services

- **Google Maps** → worker distance & navigation
- **Payment Gateway Stub** → JazzCash / Easypaisa simulation
- **Notification Service** → OneSignal / Firebase FCM
- **AI Recommendation Engine** → Custom algorithm

Development Stubs

(Used temporarily during development)

- Fake OTP Service
- Mock AI Recommendation Response
- Sample Worker Data
- Dummy Payment Response

5.3. Deployment Environment

The MERN-based system was deployed and tested in the following environment:

Local Development Environment

- **OS:** Windows 11
- **Frontend:** React on localhost:5173/3000
- **Backend:** Node.js server on localhost:5000
- **Database:** MongoDB Atlas (cloud) or MongoDB Compass (local)

Production Deployment Setup

- **Frontend Hosting:** Netlify / Vercel
- **Backend Hosting:** Render / Railway / EC2
- **Database:** MongoDB Atlas Cloud
- **Environment Variables (.env):**
    - JWT_SECRET
    - DB_CONNECTION
    - OTP_API_KEY
    - PAYMENT_GATEWAY_KEY

This ensures scalability, security, and easy deployment.

5.4. Tools and Techniques

Tools Used

- Visual Studio Code
- Postman (API debugging)
- Git & GitHub (version control)
- MongoDB Atlas Dashboard
- Figma (UI design)
- ESLint + Prettier (code formatting)
- OneSignal / Firebase for Notifications

Techniques Applied

- Component-Based Development (React)
- REST API Architecture (Express)

- MVC-like structure for server
- Modular Services and Controllers
- JWT-based Authentication
- Validation with Mongoose
- Geo-based Worker Ranking (AI Logic)
- Agile development with sprints

5.5. Best Practices / Coding Standards

- **Naming Convention:** camelCase for JS, PascalCase for components

Folder Structure: controllers, routes, models, middleware, services

Reusable Components: headers, cards, buttons, loaders

Error Handling: Centralized Express error middleware

- **Security:**

Hashed passwords

Protected routes

Environment variables

- **Database Optimization:**

Indexes on category, location

Lean queries

- F**rontend Standards:**

Avoid repeating UI components

Maintain responsive design

Use proper prop-types

5.6. Version Control

Version control for the KarigarUstaad MERN system was managed using **Git & GitHub**, following a structured branching model:

Branches

- **main** → Stable production-ready code
- **dev** → Active development

- **feature-specific branches:**
  - feature/auth
  - feature/booking
  - feature/ai-recommendation
  - feature/chat
  - feature/payment

Commit Strategy

- Small, frequent commits
- Proper commit messages (e.g., “Added JWT middleware”)

Pull Requests & Code Reviews

- All feature branches merged into dev through PRs
- Peer review used to ensure quality

Issue Tracking (GitHub Projects)

- Bugs
- Enhancements
- Pending tasks

Chapter 6

Testing and Evaluation

Chapter 6: Testing and Evaluation

This chapter presents the testing strategies and evaluation techniques applied to the KarigarUstaad system to ensure its reliability, functionality, and performance. Various software testing methods were used to validate different components of the system, including functional correctness, data handling, and system behavior under different conditions. The purpose of this chapter is to verify that the system meets the defined requirements and performs efficiently in real-world scenarios. Multiple testing approaches such as unit testing, integration testing, and performance testing were conducted to identify and resolve potential issues.

6.1. Use Case Testing

Use case testing was performed to validate the system against real user scenarios. Each use case, such as booking a service, worker accepting a job, and payment processing, was tested to ensure the system behaves as expected. This testing confirmed that all functional requirements are correctly implemented.

6.2. Equivalence partitioning

Equivalence partitioning was applied to divide input data into valid and invalid groups. Test cases were designed for each partition, such as valid login credentials, invalid email formats, and incorrect booking inputs. This reduced the number of test cases while ensuring proper validation of inputs.

6.3. Boundary value analysis

Boundary value analysis was used to test edge cases, such as minimum and maximum input values. For example, testing password length limits, booking time constraints, and rating values ensured the system handles boundary conditions without errors.

6.4. Data flow testing

Data flow testing was conducted to verify the correct movement of data within the system. It ensured that data such as user details, booking information, and payment records are properly stored, retrieved, and updated without inconsistencies.

6.5. Unit testing

Unit testing was performed on individual modules such as authentication, booking, and payment processing. Each component was tested independently to ensure it functions correctly before integration with other modules.

6.6. Integration testing

Integration testing was conducted to verify the interaction between different modules, such as frontend-backend communication and API responses. It ensured smooth data exchange between the mobile app, backend server, and database.

6.7. Performance testing

Performance testing evaluated the system's responsiveness and speed under normal conditions. The application was tested to ensure quick loading times, fast API responses, and efficient handling of multiple user requests.

6.8. Stress Testing

Stress testing was performed to analyze system behavior under extreme conditions, such as high user load or multiple simultaneous bookings. This helped identify system limitations and ensured stability during peak usage.

Chapter 7

Summary, Conclusion and Future Enhancements

Chapter 7: Summary, Conclusion & Future Enhancements

7.1. Project Summary

The KarigarUstaad project is an AI-powered home services booking platform designed to connect customers with skilled workers efficiently. The system provides a dual-mode application supporting both customers and service providers, along with an admin panel for system management. The project covers requirement analysis, system design, UI/UX development, and partial implementation using the MERN stack. Key features include service booking, worker recommendation, real-time notifications, and secure payment handling. The system aims to simplify the process of finding reliable service providers while improving accessibility and efficiency in the local service market.

7.2. Achievements and Improvements

During the development of this project, several key milestones were achieved. A complete system design was developed, including UML diagrams, database design, and architecture. The UI/UX design was finalized with a user-friendly interface for both customers and workers. Core functionalities such as booking flow, authentication, and basic backend APIs were implemented. Additionally, the AI-based recommendation concept was introduced to improve service matching. Continuous improvements were made in system design, diagram clarity, and documentation quality based on feedback and testing.

7.3. Critical Review

Although the project successfully demonstrates the core concept and system design, there are certain limitations. The implementation is partial (approximately 40%), and some advanced features such as full AI model training, real-time GPS tracking, and complete payment integration are not fully developed. The system currently relies on simplified logic for worker recommendation due to limited dataset availability. Furthermore, scalability and real-world

deployment challenges remain areas for further development. Despite these limitations, the project provides a strong foundation for a scalable and practical solution.

7.4. Lessons Learnt

This project provided valuable learning experiences in both technical and collaborative aspects. The team gained practical knowledge of the MERN stack, including frontend and backend integration. Understanding system design, UML modeling, and database structuring was significantly improved. The importance of teamwork, communication, and time management was also realized during the project lifecycle. Additionally, the project enhanced problem-solving skills, especially in handling real-world challenges such as system scalability and user experience design.

7.5. Future Enhancements/Recommendations

Several enhancements can be implemented to improve the system in future. A fully trained AI/ML model can be developed for more accurate worker recommendations using real-time data. Integration of GPS tracking can enable real-time location-based services. Secure and complete payment gateway integration can be implemented for seamless transactions. The system can be scaled to support multiple cities and a larger user base. Additional features such as multi-language support, dynamic pricing, advanced analytics, and improved security mechanisms can further enhance the platform. Expanding the B2B functionality for business clients and service providers can also increase the system's commercial viability.

Appendices

Appendix A: Information / Promotional Material

This appendix contains all promotional and branding materials created for the KarigarUstaad project. These materials support project visibility, awareness, and professional presentation. They include the brochure, flyer, standee, and banner designs prepared for project showcase, university evaluation, and public demonstration. The purpose of this appendix is to document the visual identity and marketing elements used to represent the platform in a professional and consistent manner.

A.1. Broacher

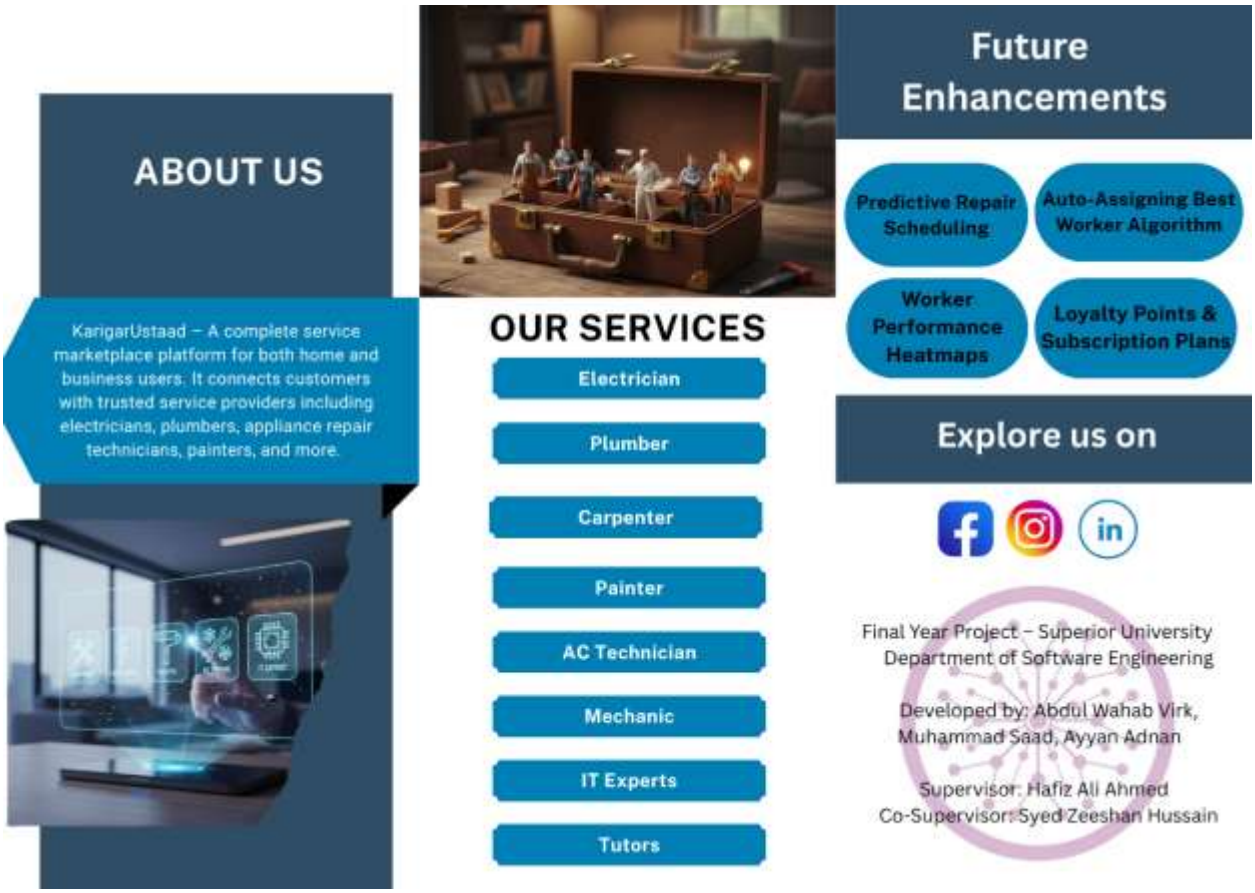


Figure 21 Broacher Front Page

Figure 22 Broacher Back Page

A.2. Flyer

Figure 23 Flyer

A.3. Standee

Figure 24 Standee

A.4. Banner

Figure 25 Banner

Reference and Bibliography

Reference and Bibliography

[1] Draw.io, *"Online Diagramming Tool,"* Software Tool, https://www.drawio.com, 2024.

[2] Mermaid.js, *"Mermaid – Diagramming and Charting Tool,"* Documentation, https://mermaid.js.org, 2024.

[3] OpenAI, *"ChatGPT – GPT-4/GPT-5 Large Language Model,"* OpenAI Documentation, San Francisco, USA, 2024.

[4] Figma Inc., *"Figma – Collaborative Interface Design Tool,"* Software Platform, https://www.figma.com, 2024.

[5] Canva, *"Canva – Online Design and Publishing Platform,"* Creative Tool, https://www.canva.com, 2024.

[6] Mahir Company, *"Official Home Services Website,"* https://mahircompany.com/ , 2024.

Index

[A] Technical Documentation

Includes all technical artifacts supporting the system analysis & design:

- Use Case Diagram
- Sequence / Collaboration Diagrams
- Class Diagram
- ERD & Data Dictionary
- State Transition Diagram
- Component Diagram
- Deployment Diagram
- Domain Model
- DFD (Data Flow Diagram)
- Operation Contracts
- Architecture Diagram (MERN)

[B] UI/UX & Prototype Screens

Includes all design and user-interface related visuals:

- Figma Wireframes (Low & High Fidelity)
- Customer Mode Screens
- Worker Mode Screens
- Admin Mode Screens (if any)
- User Flow Diagrams
- Style Guide (colors, typography)

[C] Promotional & Supporting Material

Includes all marketing and presentation content:

- Brochure
- Flyer
- Standee
- Banner
- Poster (if any)
- Branding Assets
- Logo (final & variations)
- Screenshots for social media / store listing